



**COMSATS University Islamabad,  
Sahiwal Campus**

**Apni Jgah**

**A project presented to  
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment  
of the requirement for the degree of**

***Bachelor of Science in Software Engineering (2022-2026)***

**By**

|                        |                              |
|------------------------|------------------------------|
| <b>Sawera Tubessam</b> | <b>CIIT/FA22-BSE-044/SWL</b> |
| <b>Zaid Hanif</b>      | <b>CIIT/FA22-BSE-050/SWL</b> |

# Declaration

We hereby declare that this software, neither whole nor as a part, has been copied from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proven to be copied from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted to any application for any other degree or qualification of this or any other university or institute of learning.

**Sawera Tubessam**

-----

**Zaid Hanif**

-----

# CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Apni Jgah ”was developed by **Sawera Tubessam (CIIT/FA22-BSE-044/SWL)** and **Zaid Hanif (CIIT/FA22-BSE-050/SWL)** under the supervision of “Ms. Hafsa Saleem” and co-supervisor “Dr. Javed Farzand” and that, in their opinion, it is fully adequate, in scope and quality for the degree of Bachelor of Science in Software Engineering.

-----  
**Supervisor**

**Ms. Hafsa Saleem**

Lecturer

Department of Computer Science  
COMSATS University Islamabad, Sahiwal Campus

-----  
**External Examiner**

-----  
**Head of Department**

**Dr. Zafar Iqbal Roy**

Assistant Professor

Department of Computer Science  
COMSATS University Islamabad, Sahiwal Campus

# Executive Summary

The *Apni Jgah* Real Estate Website is a web-based platform developed to simplify, modernize, and improve the process of buying, selling, and renting properties. In traditional real estate systems, property transactions are often time-consuming, inefficient, and lack transparency. Buyers usually depend on local agents or informal sources, which limits access to accurate and updated property information. Similarly, sellers face challenges in reaching a wider audience, while communication between buyers and sellers is often slow and unstructured. These limitations create difficulties in property searching, price comparison, and decision-making. To overcome these challenges, the proposed system provides a centralized and intelligent online platform that connects buyers, sellers, and administrators in a single digital environment. Users can easily browse, search, and filter properties based on their preferences, including location, price range, property type, size, and other relevant features. This significantly reduces the effort required to find suitable properties and improves the overall user experience. The platform allows property owners and agents to create and manage property listings by uploading detailed descriptions, high-quality images, pricing information, and location details. This ensures that buyers have access to complete and reliable information before making any decisions. In addition to basic functionality, the system also supports advanced features such as AI-based price prediction, which helps users estimate the fair market value of properties, and a chatbot system that provides instant assistance and guidance.

The system includes essential modules such as user registration and authentication, property listing management (add, update, delete), advanced search and filtering, favorites management, and a communication mechanism between buyers and sellers. An administrative panel is also provided to manage users, monitor property listings, and control system content such as blogs and news. This ensures that the platform remains secure, reliable, and free from fraudulent or misleading information. The website is developed using modern web technologies, ensuring a responsive and user-friendly interface that can be accessed across multiple devices, including desktops, tablets, and smartphones. The system is designed with a focus on performance, security, scalability, and usability, making it suitable for real-world implementation. Overall, the *Apni Jgah* Real Estate Website offers a comprehensive, efficient, and secure solution that transforms traditional property dealing into a smart digital experience. By improving accessibility, enhancing transparency, and integrating intelligent features, the platform aims to provide a convenient and reliable environment for all stakeholders involved in the real estate market.

# Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebted to our project supervisor “Ms. Hafsa Saleem” and our co-Supervisor “Dr. Javed Farzand”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

**Sawera Tubessam**

-----

**Zaid Hanif**

-----

# Abbreviations

| Abbreviation | Full Form                           |
|--------------|-------------------------------------|
| SRS          | Software Requirements Specification |
| PC           | Personal Computer                   |
| UI           | User Interface                      |
| DB           | Database                            |
| SQL          | Structured Query Language           |
| API          | Application Programming Interface   |
| PK           | Primary Key                         |
| FK           | Foreign Key                         |
| ERD          | Entity Relationship Diagram         |
| UML          | Unified Modeling Language           |
| AI           | Artificial Intelligence             |
| MVC          | Model View Controller               |
| SRS          | Software Requirements Specification |
| PC           | Personal Computer                   |
| UI           | User Interface                      |
| DB           | Database                            |
| SQL          | Structured Query Language           |
| API          | Application Programming Interface   |

## Table of Contents

|        |                                                                       |    |
|--------|-----------------------------------------------------------------------|----|
| 1.     | Introduction .....                                                    | 2  |
| 1.1    | Brief .....                                                           | 3  |
| 1.2    | Relevance to Course Modules .....                                     | 4  |
| 1.3    | Project Background .....                                              | 7  |
| 1.4    | Literature Review .....                                               | 8  |
| 1.5    | Analysis from Literature Review (in the context of the project) ..... | 11 |
| 1.6    | Methodology and Software Lifecycle for This Project .....             | 13 |
| 1.7    | Rationale behind Selected Methodology .....                           | 15 |
| 2.     | Problem Definition .....                                              | 18 |
| 2.1    | Problem Statement .....                                               | 19 |
| 2.2    | Deliverables and Development Requirements .....                       | 19 |
| 2.2.1  | Deliverables .....                                                    | 20 |
| 2.2.2  | Development Requirements .....                                        | 21 |
| 2.3    | Current System .....                                                  | 22 |
| 3.     | Requirement Analysis .....                                            | 25 |
| 3.1    | Use Case Diagram(s) .....                                             | 26 |
| 3.2    | Detailed Use Case .....                                               | 29 |
| 3.2.1  | User Registration .....                                               | 29 |
| 3.2.2  | Login and Authentication .....                                        | 30 |
| 3.2.3  | Search Properties .....                                               | 31 |
| 3.2.4  | Add to Favorites .....                                                | 32 |
| 3.2.5  | AI Price Prediction .....                                             | 32 |
| 3.2.6  | Filter Property .....                                                 | 33 |
| 3.2.7  | View Property Details .....                                           | 34 |
| 3.2.8  | View Favorite List .....                                              | 34 |
| 3.2.9  | Chatbot .....                                                         | 35 |
| 3.2.10 | Read News .....                                                       | 36 |
| 3.2.11 | Manage News .....                                                     | 36 |
| 3.2.12 | Manage User Account .....                                             | 37 |
| 3.2.13 | Manage Property Listing .....                                         | 38 |
| 3.2.14 | Activate/Deactivate User .....                                        | 38 |
| 3.2.15 | Delete User Account .....                                             | 39 |
| 3.2.16 | Monitor Property Listing .....                                        | 40 |

|                                                   |    |
|---------------------------------------------------|----|
| 3.3 Functional Requirements .....                 | 40 |
| 3.4 Non-Functional Requirements .....             | 45 |
| 4. Design and Architecture .....                  | 49 |
| 4.1 System Architecture .....                     | 49 |
| 4.2 Data Representation .....                     | 51 |
| 4.2.1 Database Design .....                       | 51 |
| 4.2.2 Entity-Relationship Representation .....    | 51 |
| 4.2.3 Data Flow .....                             | 53 |
| 4.2.4 Entity Descriptions and Constraints .....   | 53 |
| 4.3 Process Flow Representation .....             | 54 |
| 4.4 Design Models .....                           | 56 |
| 4.4.1 Sequence Diagram .....                      | 56 |
| 4.4.2 Class Diagram .....                         | 64 |
| 5. Implementation .....                           | 67 |
| 5.1 Algorithm .....                               | 68 |
| 5.1.1 User Registration .....                     | 68 |
| 5.1.2 Authenticate User .....                     | 68 |
| 5.1.3 Create Property Listing .....               | 69 |
| 5.1.4 Search Properties .....                     | 69 |
| 5.1.5 Get Property Details .....                  | 70 |
| 5.1.6 Add to Favourites .....                     | 70 |
| 5.1.7 AI Price Prediction .....                   | 70 |
| 5.1.8 Process Chatbot Query .....                 | 70 |
| 5.1.9 Update Property .....                       | 71 |
| 5.1.10 Delete Property .....                      | 71 |
| 5.1.11 Create Blog Post (Admin) .....             | 71 |
| 5.1.12 Send Notification .....                    | 71 |
| 5.2 External APIs .....                           | 72 |
| 5.3 User Interface .....                          | 73 |
| 6. Testing and Evaluation .....                   | 88 |
| 6.1 Manual Testing .....                          | 88 |
| 6.1.1 System Testing .....                        | 88 |
| 6.1.1.1 Purpose of System Testing .....           | 89 |
| 6.1.1.2 Types of System Testing .....             | 89 |
| 6.1.1.3 Importance of Manual System Testing ..... | 89 |



|                                                    |    |
|----------------------------------------------------|----|
| 6.1.2 Unit Testing.....                            | 89 |
| 6.1.3 Functional Testing.....                      | 90 |
| 6.1.4 Integration Testing .....                    | 91 |
| 6.2 Automated Testing .....                        | 92 |
| 7. Conclusion.....                                 | 95 |
| 7.1 Future Work.....                               | 96 |
| 7.1.1 Advanced Property Recommendations .....      | 96 |
| 7.1.2 Mobile Application Development.....          | 96 |
| 7.1.3 Virtual Property Tours.....                  | 96 |
| 7.1.4 Multi-City and International Expansion ..... | 96 |
| 7.1.5 Subscription Plans.....                      | 96 |
| 7.1.6 Advanced Analytics and Dashboard .....       | 97 |
| 7.1.7 Enhanced Security Features .....             | 97 |
| 8. References .....                                | 98 |

## List of Tables

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Table 1.1: Comparison of Existing Real Estate Applications and Apni Jgah Solution..... | 10 |
| Table 3.1: User Registration .....                                                     | 30 |
| Table 3.2: Login and Authentication.....                                               | 31 |
| Table 3.3: Search Properties .....                                                     | 31 |
| Table 3.4: Favorites.....                                                              | 32 |
| Table 3.5: AI price prediction .....                                                   | 33 |
| Table 3.6: Filter Property .....                                                       | 33 |
| Table 3.7: View Property Details .....                                                 | 34 |
| Table 3.8: View Favorites .....                                                        | 35 |
| Table 3.9: Chatbot.....                                                                | 35 |
| Table 3.10: Read News .....                                                            | 36 |
| Table 3.11: Update News .....                                                          | 37 |
| Table 3.12: Manage User Account.....                                                   | 37 |
| Table 3.13: Update Property .....                                                      | 38 |
| Table 3.14: Activate/Deactivate User.....                                              | 39 |
| Table 3.15: Delete User Account .....                                                  | 39 |
| Table 3.16: Monitor Property Listing .....                                             | 40 |
| Table 3.17: User Authentication .....                                                  | 41 |
| Table 3.18: Property Listing Management.....                                           | 42 |
| Table 3.19: Property Search and Filtering .....                                        | 42 |
| Table 3.20: AI-Based Price Prediction.....                                             | 43 |
| Table 3.21: Favorites Management.....                                                  | 43 |
| Table 3.22: AI Chatbot Support .....                                                   | 44 |
| Table 3.23: Blogs and News Section .....                                               | 44 |
| Table 3.24: Admin Dashboard .....                                                      | 45 |
| Table 5.1: External APIs .....                                                         | 72 |
| Table 6.1: Unit Testing.....                                                           | 90 |
| Table 6.2: Functional Testing.....                                                     | 91 |
| Table 6.3: Integration Testing .....                                                   | 92 |
| Table 6.4: Automated Testing .....                                                     | 93 |

## List of Figures

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Figure 3.1:Apni Jgah Use Case Diagram.....                                 | 28 |
| Figure 4.1: Three-Tier Architecture Diagram for Real Estate Platform ..... | 50 |
| Figure 4.2:Entity–Relationship Diagram of Apni Jgah System.....            | 52 |
| Figure 4.3 :Detailed Table Relationship Structure in SQLite .....          | 53 |
| Figure 4.4: Process Flow Diagram for Apni Jgah.....                        | 55 |
| Figure 4.5: Buyer Property Search and Operations Sequence Diagram.....     | 57 |
| Figure 4.6: Seller Create and Listing of Properties Sequence Diagram ..... | 59 |
| Figure 4.7: Admin Operations Sequence Diagram .....                        | 61 |
| Figure 4.8: AI Chatbot Query Processing Sequence Diagram.....              | 63 |
| Figure 4.9: Class Diagram .....                                            | 65 |
| Figure 5.1:Homepage.....                                                   | 74 |
| Figure 5.2: Login Page.....                                                | 75 |
| Figure 5.3:Search Property .....                                           | 76 |
| Figure 5.4: Property Browsing Page.....                                    | 77 |
| Figure 5.5:Chatbot .....                                                   | 77 |
| Figure 5.6: Chatbot Query .....                                            | 78 |
| Figure 5.7: Professional Workers.....                                      | 78 |
| Figure 5.8: Professionals Profiles .....                                   | 79 |
| Figure 5.9: My Property Page.....                                          | 79 |
| Figure 5.10: 3D Virtual Tour.....                                          | 80 |
| Figure 5.11: Contact Screen.....                                           | 81 |
| Figure 5.12: News Screen .....                                             | 81 |
| Figure 5.13: About Us.....                                                 | 82 |
| Figure 5.14: Favorites .....                                               | 83 |
| Figure 5.15: Buyer Profile .....                                           | 84 |
| Figure 5.16: Seller Profile.....                                           | 85 |
| Figure 5.17: Professional Profile .....                                    | 86 |

# **Chapter 1**

## **INTRODUCTION**

# 1. Introduction

The Apni Jgah – AI Based Real Estate Price Prediction & Property Finder project is designed as a comprehensive and centralized real estate management system that connects buyers, sellers, and administrators through a unified digital platform. The primary objective of this system is to modernize the traditional property dealing process by integrating advanced technologies, improving accessibility, and providing intelligent decision-making support. In conventional real estate practices, property transactions are often carried out manually through agents or fragmented online sources, where information is either incomplete, outdated, or difficult to verify. These limitations create challenges for users, including lack of transparency, inefficient communication, and increased risk of poor investment decisions.

The Apni Jgah platform addresses these challenges by offering a centralized environment where all real estate activities can be managed efficiently. It provides users with a single platform to explore, analyze, and interact with property listings in a structured and reliable manner. Buyers can easily search for properties based on their preferences such as location, price range, size, and type, while sellers can list and manage their properties with complete control. Administrators oversee the system to ensure data authenticity, monitor activities, and maintain system integrity. This centralized approach eliminates the need for multiple platforms and reduces dependency on traditional intermediaries, making the process more transparent and efficient. One of the major innovations introduced in this project is the integration of Artificial Intelligence (AI) to enhance the overall functionality of the system. Unlike traditional platforms that only display property listings, Apni Jgah incorporates an intelligent price prediction mechanism that helps users evaluate whether a property is fairly priced according to current market conditions. The AI model analyzes various factors such as historical property data, location trends, property features, and market demand to estimate the expected value of a property. This predictive capability provides users with valuable insights, enabling them to make informed and confident decisions when buying or investing in real estate. The inclusion of AI not only improves decision-making but also adds a level of automation and intelligence that is often missing in conventional systems. For instance, users no longer need to rely solely on agents or personal judgment to assess property value. Instead, they can use the system's predictions as a reliable reference point. This feature is particularly beneficial for new investors or individuals who lack extensive knowledge of the real estate market. By reducing uncertainty and providing data-driven insights, the system contributes to a more efficient and trustworthy property marketplace.

In addition to AI-based price prediction, the Apni Jgah platform offers a range of features designed to enhance user experience and usability. The system includes advanced search and filtering options that allow users to quickly find properties that match their specific requirements. Detailed property pages provide comprehensive information, including images, descriptions, pricing, and location details, enabling users to evaluate options without the need for physical visits in the initial

stages. The platform also supports Wishlist functionality, allowing users to save and revisit their preferred properties, thereby improving convenience and decision-making. Furthermore, the system incorporates modern communication and support features such as chatbot assistance, which provides instant responses to user queries. This improves interaction and reduces response time compared to traditional systems where users often have to wait for agent feedback. The inclusion of a blog section also adds value by providing educational content related to real estate trends, investment strategies, and market insights. This helps users stay informed and enhances their understanding of the property market.

From a technical perspective, the Apni Jgah system is developed using modern web technologies, ensuring scalability, reliability, and performance. The platform follows a client-server architecture where the frontend interface communicates with the backend server to process user requests and manage data efficiently. The use of structured databases ensures that all property and user information is stored securely and can be retrieved quickly when needed. Security measures such as authentication and authorization are implemented to protect user data and ensure safe system usage. Another important aspect of the project is its focus on improving transparency and reducing fraud in the real estate sector. By providing verified property listings and centralized data management, the system minimizes the chances of misleading information. Users can rely on accurate and up-to-date data, which builds trust and confidence in the platform. The administrative controls further ensure that inappropriate or fraudulent listings are monitored and removed promptly. Overall, the Apni Jgah project represents a significant advancement over traditional real estate systems by combining digital accessibility with intelligent analytics. It transforms the way users interact with property markets by making the process faster, smarter, and more reliable. The integration of AI, along with user-friendly design and robust backend support, makes the platform a complete solution for modern real estate needs.

Apni Jgah system aims to bridge the gap between traditional property dealing methods and modern technological advancements. By providing a centralized, intelligent, and user-oriented platform, it simplifies property searching, enhances decision-making, and improves overall efficiency in the real estate sector. This project not only fulfills academic requirements but also has the potential to be developed into a real-world application that can significantly impact the property market.

## **1.1 Brief**

The Apni Jgah project is a centralized real estate management system designed to connect buyers, sellers, and administrators through a unified digital platform. It aims to address the limitations of traditional real estate methods where property information, pricing details, and communication are often scattered, manual, and unreliable. By bringing all essential services into one system, Apni Jgah ensures secure data handling, easy access to information, and smooth interaction between all users. The platform is developed using modern web technologies along with Artificial Intelligence (AI) to provide a scalable, efficient, and user-friendly property management experience.

The main purpose of the system is to modernize and simplify real estate operations by offering an all-in-one solution for property searching, listing, price prediction, and communication. It reduces dependency on traditional agents and manual processes while empowering users with intelligent features such as AI-based price prediction and market trend analysis. These features help users make informed decisions when buying, selling, or investing in properties. The system also improves user experience through chatbot assistance, favorites management, and real-time interaction features.

The scope of Apni Jgah includes a complete web-based platform that supports buyers, sellers, and administrators. Buyers can search and filter properties, view detailed listings, save favorites, and directly contact sellers. Sellers can manage their property listings by adding, updating, or deleting properties and uploading images or 3D views. The system also includes AI-powered modules for price prediction and market analysis, along with additional features such as a chatbot, wishlist functionality, and a blog section for real estate updates. The admin panel ensures proper system management by approving listings, managing users, and moderating content.

Technically, the system is developed using Django for backend development, while HTML, CSS, and JavaScript are used for the frontend interface. MySQL is used for database management to store user data, property listings, and system records. Artificial Intelligence techniques are integrated to analyze property data and generate price predictions, making the platform more intelligent and data-driven.

Overall, Apni Jgah functions as an integrated real estate ecosystem where users can efficiently manage property-related activities. Buyers can easily explore and evaluate properties, sellers can effectively manage listings, and administrators can maintain system integrity. The inclusion of AI-based insights and modern features significantly improves transparency, reduces manual effort, and enhances decision-making. As a result, the system provides a secure, reliable, and efficient solution for modern real estate needs.

## **1.2 Relevance to Course Modules**

The development of the Apni Jgah project is strongly connected with the core subjects studied throughout the Bachelor of Software Engineering program. Each phase of the project, including requirement analysis, system design, implementation, and testing, reflects the practical application of theoretical knowledge gained during the degree. The project demonstrates how different course modules contribute collectively to building a complete, intelligent, and real-world real estate platform.

Apni Jgah is not just a simple web application but an integrated system that combines web development, database management, artificial intelligence, and user experience design. The

following sections describe how each major course module contributed to the successful development of this project.

## **Software Engineering**

Software Engineering played a fundamental role in structuring the overall development process of the Apni Jgah system. Concepts such as requirement gathering, feasibility analysis, system modeling, and documentation were applied to clearly define the system scope and functionalities. The project follows a structured Software Development Life Cycle (SDLC), where each phase planning, analysis, design, implementation, and testing was carried out systematically. Use case diagrams, functional and non-functional requirements, and system design models were developed based on the principles learned in this course. These practices ensured that the system is well-organized, scalable, and easy to maintain.

## **Database Systems**

The Database Systems course provided the foundation for designing and managing the data layer of the Apni Jgah platform. A relational database (SQLite/MySQL) is used to store and manage data such as user profiles, property listings, transactions, and Wishlist records. Concepts such as database normalization, relationships, indexing, and query optimization were applied to ensure efficient data storage and retrieval. The database design ensures data consistency, integrity, and security, which are essential for handling real estate information reliably.

## **Artificial Intelligence / Machine Learning**

Artificial Intelligence and Machine Learning concepts are a key highlight of the Apni Jgah system. These concepts were applied to develop a property price prediction model, which analyzes historical data and market trends to estimate property values. The system uses data analysis techniques to identify patterns based on features such as location, size, and property type. This intelligent feature helps users make informed decisions and adds significant value compared to traditional real estate platforms. The integration of AI demonstrates how modern technologies can enhance user experience and decision-making.

## **Web Development**

The development of the Apni Jgah web application is directly supported by the concepts learned in Web Engineering and Internet Programming. Technologies such as HTML, CSS, Bootstrap, JavaScript, and Django were used to build a dynamic and responsive system. Frontend development focuses on creating an interactive and user-friendly interface, while backend development using Django manages business logic, database interaction, and API handling.



Concepts such as routing, form handling, session management, and RESTful communication were implemented to ensure smooth interaction between the user interface and the server.

### **Data Structures and Algorithms**

Data Structures and Algorithms were applied to optimize the performance of the system, especially in searching, sorting, and filtering operations. Efficient algorithms are used to retrieve property data based on user queries such as location, price range, and property type. These concepts ensure that the system can handle large amounts of data efficiently without performance issues. The use of optimized logic improves response time and enhances the overall user experience

### **Human Computer Interaction**

Human Computer Interaction (HCI) principles were used to design a user-friendly and visually appealing interface for the Apni Jgah platform. The system is designed with a focus on simplicity, clarity, and ease of navigation.

Features such as structured layouts, clear buttons, intuitive navigation menus, and responsive design ensure that users can interact with the system easily. The goal is to provide a smooth and satisfying user experience for both technical and non-technical users.

### **Object-Oriented Programming**

Object-Oriented Programming (OOP) concepts played a crucial role in organizing the system structure. The use of classes, objects, inheritance, and encapsulation helped in building modular and reusable components within the system. In Django, models, views, and controllers are structured using OOP principles, which makes the system more maintainable and scalable. This approach ensures that different modules such as user management, property handling, and recommendation systems are well-separated and easy to manage.

### **Information Security**

Information Security concepts were applied to protect user data and ensure secure system operations. Features such as authentication, authorization, password encryption, and session management were implemented to prevent unauthorized access. User data, including personal information and property details, is handled securely to maintain privacy and trust. These security measures are essential for any modern web-based system.

### **Computer Networks**

The Apni Jgah system operates on a client-server architecture, where the frontend communicates with the backend through HTTP requests. Concepts from Computer Networks helped in

understanding how data is transmitted between the client and server. API integration, request handling, and server communication are based on networking principles. These concepts ensure that the system performs efficiently in a web-based environment.

## **Final Year Project I and II**

The Final Year Project courses provided the overall framework and guidance for developing the Apni Jgah system. These courses helped in planning, designing, documenting, and implementing the project in a structured manner. Through different phases such as proposal, design evaluation, and final implementation, the project was continuously improved based on feedback. These courses ensured proper documentation, time management, and professional presentation of the project.

### **1.3 Project Background**

The real estate industry has witnessed rapid growth and transformation over the past decade due to increasing urbanization, population growth, and rising demand for residential and commercial properties. In countries like Pakistan, the property market plays a significant role in economic development, attracting both local and overseas investors. With the advancement of internet technologies, many online real estate platforms have emerged that allow users to buy, sell, and rent properties from the comfort of their homes. These platforms have simplified access to property information by providing digital listings, images, location details, and basic filtering options.

However, despite these advancements, most existing real estate systems still suffer from several limitations. The majority of platforms primarily focus on displaying property listings without offering intelligent decision-making support. Users are often required to manually evaluate properties based on incomplete or unverified information. One of the most critical challenges faced by users is determining whether the price of a property is fair according to current market conditions. Property prices are influenced by multiple factors such as location, size, demand, economic trends, and nearby facilities, making it difficult for buyers especially inexperienced ones to make accurate judgments.

As a result, users may end up making poor investment decisions, either by overpaying for a property or missing better opportunities. Additionally, there is a lack of personalized recommendations, meaning users are not guided toward properties that best match their preferences and budget. Traditional platforms also lack intelligent interaction, real-time assistance, and advanced analytical features that could improve user experience and decision-making.

With the rapid evolution of modern technologies, particularly Artificial Intelligence (AI) and Data Analytics, it has become possible to overcome these limitations. AI techniques can analyze large volumes of historical property data, identify patterns, and predict future trends in the real estate

market. Machine learning models can evaluate factors such as location, property type, price history, and market demand to generate accurate property price predictions. These insights can significantly help users in making informed and confident decisions.

The concept of integrating AI into real estate systems has opened new opportunities for developing smart platforms that go beyond traditional listing services. By combining intelligent algorithms with user-friendly interfaces, modern systems can provide personalized recommendations, price estimations, and automated assistance through chatbots.

The Apni Jgah platform is developed as a smart real estate solution that addresses the limitations of existing systems by incorporating advanced technologies into a single, unified platform. It not only allows users to search, buy, sell, and rent properties but also enhances the experience by providing AI-based price prediction features. The system analyzes relevant property data and offers estimated prices, helping users understand whether a property is reasonably priced.

In addition to this, the platform includes several modern features such as:

- Favorites or Wishlist management, allowing users to save preferred properties
- AI-powered chatbot support for instant assistance and queries
- Real estate blogs and articles to educate users about market trends and investment strategies
- Advanced filtering and search options for better usability

These features collectively improve the efficiency, transparency, and reliability of the real estate process.

The primary aim of Apni Jgah is to provide a comprehensive, intelligent, and user-friendly platform that simplifies property searching and decision-making. By integrating AI and modern web technologies, the system ensures that users can access accurate information, receive smart recommendations, and make better investment choices. Ultimately, the platform contributes toward transforming the traditional real estate experience into a more digital, data-driven, and efficient process.

## **1.4 Literature Review**

The reviewed literature shows that several real estate applications and property platforms provide features such as property listing, buying, selling, and renting services. These platforms help users explore available properties and connect with sellers or agents. However, most of these systems are limited to basic functionalities and do not provide intelligent support for decision-making. They mainly act as listing platforms where users manually search and compare properties without any advanced assistance.

A major limitation found in existing systems is the lack of updated and reliable information. In many cases, property data is either outdated or incomplete, which creates confusion for users. Additionally, most platforms do not provide any form of price analysis or guidance, meaning users are unable to determine whether a property is fairly priced according to market conditions. This makes the decision-making process difficult, especially for new users who do not have prior experience in the real estate market.

Another common issue identified in the literature is the lack of proper comparison and filtering tools. Although some platforms offer search options, they are often basic and do not allow users to filter properties effectively based on multiple conditions such as budget, location, size, and property type at the same time. This results in a time-consuming process where users have to manually review many listings before making a decision. Furthermore, personalization features are also missing in most systems, which means users are not guided based on their preferences or past activity.

User experience is another important area where existing systems show weaknesses. Many platforms have complex interfaces or overloaded information, which makes them difficult to navigate for non-technical users. In addition, there is a lack of interactive features such as chat support, recommendations, or intelligent suggestions. Most systems also do not provide centralized and well-organized data, which further reduces efficiency and usability.

To clearly understand these limitations and how they are addressed, a comparison between existing platforms and the proposed Apni Jgah system is presented below:

- Zameen.pk provides property listings but does not include simple price guidance. Users can browse properties, but they are not assisted in understanding whether the price is reasonable. Apni Jgah improves this by introducing a basic price estimation feature that gives users an idea of property value based on simple analysis, helping them make better decisions.
- OLX Property allows users to post and view property listings, but the data is not always verified and filtering options are limited. This often leads to irrelevant or untrustworthy listings. In contrast, Apni Jgah provides structured and organized property data with improved filtering options, allowing users to search properties based on multiple criteria such as price range, location, and type more effectively.
- Graana.com mainly focuses on property marketing and listing services but lacks interactive support for users. Apni Jgah addresses this limitation by integrating a simple chatbot feature that provides basic assistance and answers common user queries, improving overall user interaction and guidance.
- Local property dealers still rely on manual processes, which are time-consuming and inefficient. There is no proper digital record system, and communication depends on physical visits or phone calls. Apni Jgah replaces this traditional approach by offering a

fully digital platform where all property-related activities are managed online in a centralized system, making the process faster and more efficient.

- Facebook Marketplace allows users to post property listings, but the information is often unstructured and lacks verification. This increases the risk of fake or incomplete data. Apni Jgah improves reliability by maintaining organized and properly managed property records within a structured database, ensuring better data quality and trustworthiness.
- Basic real estate applications generally only provide listing functionality without any additional smart features. They do not support features such as saving favorite properties or reading informative content. Apni Jgah enhances user experience by including a Wishlist feature, a blog section for real estate knowledge, and simple intelligent tools that help users in decision-making.

Overall, the literature clearly highlights that while existing real estate platforms are useful for basic property searching and listing, they lack intelligence, organization, and user support features. The Apni Jgah system addresses these gaps by combining structured data management, user-friendly design, and simple intelligent features in a single platform, making it more effective, practical, and suitable for modern real estate needs.

**Table 1.1: Comparison of Existing Real Estate Applications and Apni Jgah Solution**

| <b>Application Name</b> | <b>Weakness</b>                                                              | <b>Proposed Project Solution (Apni Jgah)</b>                                       |
|-------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Zameen.pk               | Provides property listings but lacks simple price guidance for users.        | Apni Jgah provides basic price estimation to help users understand property value. |
| OLX Property            | Listings are not always verified and lack detailed filtering.                | Apni Jgah offers structured listings with better filtering options.                |
| Graana.com              | Focuses mainly on listings and marketing, limited user assistance features.  | Apni Jgah includes simple chatbot support for user guidance.                       |
| Local Property Dealers  | Manual system, no digital records, and time-consuming process.               | Apni Jgah provides a complete digital platform for property management.            |
| Facebook Marketplace    | Unstructured listings and high chances of fake or incomplete data.           | Apni Jgah ensures organized and managed property data.                             |
| Basic Real Estate Apps  | Only provide listings without additional features like Wishlist or insights. | Apni Jgah includes Wishlist, blogs, and simple smart features in one system.       |

## **1.5 Analysis from Literature Review (in the context of the project)**

From the literature review, it is clear that existing real estate platforms mainly focus on property listing and searching. While these platforms provide useful information, they lack advanced intelligent features that can help users make better investment decisions. Research studies show that machine learning algorithms can effectively predict property prices based on historical data and market trends. However, many commercial property websites do not fully utilize these technologies.

The Apni Jgah system aims to address this gap by integrating AI-based price prediction into the property search platform. In addition, the system includes features such as chatbot assistance, favorites management, and property blogs to improve user experience.

By combining these technologies and features, the platform provides a more intelligent and user-friendly real estate solution compared to traditional systems.

### **Alignment with Digital Trends**

The literature clearly indicates that many industries are shifting from traditional manual systems to digital platforms. This transformation is mainly driven by the need for efficiency, accessibility, and better management of information. In the real estate sector, this shift is also very visible as property dealing is gradually moving from physical offices and agents to online platforms. Digital real estate systems allow users to search for properties, compare options, and make decisions without visiting multiple locations. This not only saves time but also improves convenience for users. The increasing use of smartphones and internet access has further accelerated the adoption of online property platforms. Apni Jgah aligns with these modern digital trends by providing a complete web-based platform where all real estate activities can be performed in one place. Users can browse properties, view details, and manage their preferences without relying on traditional methods. This alignment shows that the system is designed according to current technological advancements and user needs.

### **Comparison with Existing Platforms**

The literature review shows that existing real estate platforms provide basic services such as property listings and search functionality. While these features are useful, they are not sufficient to fully support users in making decisions. Many platforms lack advanced filtering, proper guidance, and interactive features that enhance user experience. Apni Jgah improves upon these limitations by offering better filtering options that allow users to search properties based on specific requirements such as price range, location, and type. It also introduces a simple price guidance feature that helps users understand whether a property is reasonably priced. In addition, the system provides a clean and user-friendly interface that makes navigation easy even for non-technical users. By improving these aspects, Apni Jgah becomes more practical and helpful

compared to existing platforms, as it focuses not only on listing properties but also on improving user experience.

### **Improvement in Property Search**

Searching for properties in traditional systems and even in some online platforms can be a time-consuming process. This is mainly because information is often scattered, unorganized, or incomplete. Users may need to visit multiple websites or contact different agents to find suitable options. Apni Jgah addresses this issue by providing a structured and organized system for property listings. All properties are stored in a centralized database, which allows users to access complete and consistent information. The system also provides filtering options that help users narrow down their search based on their preferences. This improvement reduces the effort required by users and makes the property search process faster and more efficient. Users can easily compare multiple properties and make better decisions without confusion.

### **Simple Price Guidance Feature**

One of the major gaps identified in the literature is the lack of price understanding in existing real estate platforms. Most systems only display property prices without providing any indication of whether the price is reasonable according to market conditions. Apni Jgah introduces a simple price guidance feature that gives users an estimated idea of property value. This feature is designed to be easy to understand and does not involve complex calculations. It helps users get a general understanding of pricing, which is especially useful for beginners who may not have experience in the real estate market. By providing this basic level of guidance, the system supports better decision-making while keeping the functionality simple and user-friendly.

### **User-Friendly Features**

The literature shows that many existing platforms focus only on core functionality and do not include additional features that improve user experience. This makes the system less interactive and sometimes difficult to use. Apni Jgah addresses this issue by including several user-friendly features that enhance usability. The Wishlist feature allows users to save properties they are interested in, making it easier to revisit and compare options later. The blog section provides simple and useful information about real estate, helping users gain basic knowledge about the market. The chatbot feature offers quick assistance by answering common user queries. These features make the platform more interactive and convenient, improving the overall experience for users.

### **Addressing Existing Limitations**

The literature identifies several limitations in current real estate systems, including lack of guidance, unorganized data, and reliance on manual processes. These issues create difficulties for users and reduce the efficiency of the system. Apni Jgah is designed to overcome these limitations by providing a centralized platform where all data is organized and easily accessible. The system reduces dependency on manual processes by offering digital solutions for property management.

It also includes simple smart features that guide users and improve decision-making. By addressing these issues, Apni Jgah provides a more complete and efficient solution compared to traditional and existing systems. It focuses on simplicity, usability, and practical functionality, making it suitable for real-world applications.

## **1.6 Methodology and Software Lifecycle for This Project**

The development of the Apni Jgah – AI Based Real Estate Price Prediction & Property Finder project follows a structured approach based on software engineering principles and a suitable Software Development Life Cycle (SDLC) model. The system includes multiple features such as property listing, user management, search functionality, Wishlist, and basic price estimation. Therefore, a proper methodology is required that supports step-by-step development, continuous improvement, and effective management of different modules. Instead of building the entire system at once, the project is developed in smaller parts so that each feature can be designed, implemented, tested, and improved individually. This approach helps in reducing errors, improving system quality, and ensuring that all user requirements are properly fulfilled. By following a structured development process, the system becomes more organized, reliable, and easier to maintain.

The project adopts the Agile methodology, which focuses on iterative and incremental development. Agile is suitable for this system because it allows continuous feedback and flexible changes during development. Requirements can be updated at any stage, which makes it ideal for a project where improvements and enhancements are expected. The system is developed in small cycles, where each cycle focuses on a specific feature or module. For example, user authentication including registration and login is developed first, followed by property listing and search functionality. Later, advanced features such as filtering, Wishlist, blog section, and price estimation are added step by step. After completing each part, it is tested to ensure proper functionality before moving to the next stage.

The development process follows an iterative SDLC model, where the system goes through repeated phases of planning, designing, development, testing, and improvement. Each iteration focuses on a specific set of requirements, ensuring that every module is properly developed and integrated into the system. This approach is beneficial for Apni Jgah because the system consists of multiple interconnected components such as user management, property handling, and search operations. By using iterative development, each component is carefully built and validated, which reduces the risk of errors and improves overall system stability.

All system requirements are managed in a structured product backlog, which contains a complete list of features to be implemented in the system. These include user authentication, property listing, advanced search filters, Wishlist functionality, blog section, and simple price guidance feature. The items in the backlog are prioritized based on their importance, and development is carried out



accordingly. As the project progresses, the backlog is continuously updated to include new requirements or improvements suggested during testing or feedback sessions. This ensures that development remains focused, organized, and aligned with user needs.

The system is also developed using incremental delivery, where each feature is completed and delivered step by step instead of releasing the full system at once. Initially, basic features such as user registration and login are implemented. After that, property management and search functionalities are added. Later, additional features like Wishlist, filtering system, blog section, and price estimation are integrated. Each increment is tested individually before combining it with the rest of the system. This method makes testing easier and ensures that every part of the system functions correctly before final integration.

The Agile methodology is chosen for this project because it provides flexibility, simplicity, and better control over development. It allows changes to be made easily without affecting the entire system, which is important for a project that evolves through feedback and improvements. Since Apni Jgah includes multiple modules but is not overly complex, Agile provides the right balance between structure and adaptability. It also ensures continuous testing and improvement, which helps in delivering a high-quality system that meets user expectations.

For development, simple and widely used technologies are selected to ensure efficiency and ease of implementation. The backend is developed using Django, which handles data processing, business logic, and system operations. The frontend is created using HTML, CSS, and Bootstrap to provide a responsive and user-friendly interface. The database is managed using SQLite or MySQL, where all user and property data is stored in an organized manner. These technologies are reliable and suitable for building web-based applications, making the system efficient and scalable.

Testing is an essential part of the development process in Apni Jgah. Each feature is tested after development to ensure it works correctly and meets the required functionality. Different testing methods are used to check user authentication, property search, filtering, Wishlist operations, and price estimation feature. Any errors or issues identified during testing are fixed immediately before proceeding to the next stage. Continuous testing ensures that the system remains stable, functional, and user-friendly throughout the development process.

The selected Agile-based SDLC approach offers several advantages for this project. It enables development in small, manageable steps, making the process easier to control and understand. Errors can be identified and fixed early, reducing the chances of major issues in later stages. It also supports continuous improvement through feedback, ensuring that the system evolves according to user needs. Overall, this methodology improves collaboration, enhances system quality, and results in a more reliable and efficient real estate platform that fulfills all required functionalities effectively.

## 1.7 Rationale behind Selected Methodology

Agile methodology was selected for the development of the *Apni Jgah* real estate platform because it provides a flexible, iterative, and highly adaptive approach to software development. This methodology is particularly suitable for complex and evolving systems where requirements cannot be fully defined at the initial stage. In contrast to traditional models such as the Waterfall model, where all requirements must be finalized before development begins, Agile allows continuous refinement and modification of requirements throughout the project lifecycle. This flexibility is essential for this project, as features such as property listing, AI-based price prediction, chatbot interaction, and admin management may evolve based on user feedback, testing outcomes, and real-world usage scenarios.

One of the most significant advantages of Agile is its iterative development process, which divides the project into small, manageable units called sprints. Each sprint focuses on developing a specific feature or module, such as user authentication, property management, search and filtering, or favorites functionality. At the end of each sprint, a working version of the feature is delivered. This incremental approach ensures that the system is built step by step, making it easier to track progress, identify issues early, and maintain high development quality. It also reduces the risk of major system failures, as problems are detected and resolved at an early stage.

Agile methodology also promotes continuous stakeholder involvement and collaboration, which is crucial for developing a user-centered system like *Apni Jgah*. Regular interactions with stakeholders, including supervisors, developers, and potential users, allow the development team to gather feedback after each iteration. This ensures that the system remains aligned with user expectations and business requirements. For example, features like advanced search filters, AI recommendations, and chatbot responses can be refined based on actual user needs, improving overall usability and satisfaction.

Another important aspect of Agile is its emphasis on continuous testing and integration. Unlike traditional approaches where testing is performed only after development is completed, Agile integrates testing into every stage of the development process. Each module is tested individually during its development cycle, ensuring that errors, bugs, and inconsistencies are identified and fixed early. This leads to improved system reliability and reduces the chances of critical failures in later stages. It also supports smooth integration of different system components, such as the backend database, frontend interface, and external APIs.

Agile also enhances adaptability to change, which is particularly important for this project due to its multiple interconnected modules. For instance, if there is a need to modify the property data structure, update the AI prediction model, or improve the recommendation system, these changes can be implemented without disrupting the entire system. Agile allows developers to quickly respond to new requirements, making the development process more efficient and less rigid.

Furthermore, Agile encourages better team collaboration and communication. Developers, designers, and testers work closely together throughout the project, ensuring that all components of the system are aligned and integrated effectively. Daily or weekly meetings (such as stand-ups or sprint reviews) help track progress, discuss challenges, and plan future tasks, resulting in improved coordination and productivity.

In addition, Agile supports faster delivery of functional software. Since each sprint produces a working module, the system can be partially deployed or demonstrated at early stages. This is beneficial for academic projects, as it allows continuous evaluation and improvement before final submission.

Overall, Agile methodology provides a structured yet flexible framework that supports continuous improvement, efficient development, and high-quality output. Its iterative nature, combined with strong focus on user feedback, testing, and adaptability, makes it an ideal choice for developing the *Apni Jgah* real estate platform, which requires dynamic features, scalability, and a user-friendly experience.

# **Chapter 2**

## **PROBLEM DEFINITION**

## 2. Problem Definition

This chapter provides a comprehensive and detailed explanation of the core problems addressed by the Apni Jgah real estate platform. The real estate sector in Pakistan, despite its rapid growth and economic importance, still faces numerous challenges due to its reliance on traditional and unstructured methods of operation. These challenges not only affect buyers and sellers but also reduce overall market efficiency and transparency.

One of the most significant problems in the current real estate environment is the lack of a centralized and reliable digital platform. Property-related information is scattered across multiple sources such as social media platforms, local property dealers, WhatsApp groups, newspaper advertisements, and small informal websites. This fragmentation creates confusion for users, as they are required to search through multiple channels to find relevant property information. As a result, users often encounter duplicate listings, outdated data, incomplete property descriptions, and inconsistent pricing information.

Another major issue is the heavy dependency on intermediaries such as property agents and brokers. While these agents play an important role in facilitating transactions, they also introduce several inefficiencies into the system. The involvement of intermediaries increases transaction costs, reduces transparency, and often leads to delays in communication between buyers and sellers. In many cases, users are unable to directly interact with property owners, which limits their ability to negotiate effectively and make timely decisions.

Trust and security are also critical concerns in the real estate sector. Due to the absence of proper verification mechanisms, many listings are either fake or misleading. Users frequently face issues such as incorrect property details, false pricing, and unverified ownership. This lack of trust discourages users from engaging confidently in property transactions and increases the risk of fraud.

Furthermore, the current system lacks the integration of modern technologies that could significantly improve user experience and decision-making. Features such as artificial intelligence, data-driven recommendations, predictive analytics, and automated filtering are either missing or very limited. Users are forced to manually browse through large volumes of data, which is both time-consuming and inefficient. This is particularly challenging for users who are not familiar with the real estate market.

Communication within the current system is also unstructured and inefficient. Most interactions between buyers and sellers occur through phone calls, messages, or intermediaries, which can lead to misunderstandings and delays. There is no real-time communication system or centralized platform where users can securely exchange information, track inquiries, or receive updates.

In addition, there is a significant gap between urban and rural real estate markets. While some urban areas have access to online platforms, many smaller cities and rural regions still rely entirely on traditional methods. This creates inequality in access to property information and limits opportunities for users in less developed areas.

Overall, these challenges highlight the urgent need for a centralized, intelligent, and user-friendly real estate platform that can address issues of transparency, efficiency, reliability, and accessibility. The Apni Jgah system is designed to overcome these limitations by providing a structured, secure, and technology-driven solution for modern real estate management.

## **2.1 Problem Statement**

The real estate industry in Pakistan faces multiple critical challenges due to the absence of a centralized and intelligent system. Property data is often unstructured, inconsistent, and scattered across different sources, making it difficult for users to access reliable and updated information. Buyers struggle to identify authentic properties, compare options effectively, and make informed decisions, while sellers face difficulties in reaching genuine and interested buyers.

The current system heavily relies on manual processes and intermediaries, which increases cost, reduces transparency, and slows down the transaction process. In addition, the lack of verification mechanisms leads to fraudulent listings and trust issues among users. Communication between stakeholders is also inefficient, as there is no structured or real-time interaction system.

To address these challenges, the Apni Jgah project aims to develop a centralized and intelligent real estate web platform. The system provides verified property listings, advanced search and filtering options, AI-based price estimation, and secure communication between users. It also includes dashboards for sellers and administrators to manage listings and monitor system activity.

By integrating modern technologies and user-friendly features, the platform enhances transparency, reduces dependency on intermediaries, and improves the overall efficiency and reliability of real estate transactions. The proposed solution aims to create a trustworthy and efficient environment for all stakeholders involved in the property market.

## **2.2 Deliverables and Development Requirements**

The Apni Jgah system is designed as a complete digital real estate management solution. This section describes the expected deliverables and technical requirements necessary for system development.

### 2.2.1 Deliverables

The Apni Jgah project delivers a complete real estate solution that covers all essential functionalities required in a modern property management system. The first major deliverable is a fully responsive user web application. This application is designed to work smoothly on desktops, tablets, and mobile devices, allowing users to easily search and explore properties. Users can apply filters such as location, price range, property type, number of rooms, and availability status to find suitable properties. Each property listing includes detailed information such as high-quality images, descriptions, price, location details, and contact information of the seller or agent. This ensures that users can make informed decisions without needing external assistance.

Another important deliverable is the seller dashboard, which provides property owners and real estate agents with a dedicated interface to manage their listings. Through this dashboard, sellers can add new properties by entering complete details such as title, description, price, location, and uploading multiple images. They can also update existing listings, remove outdated properties, and track the performance of their listings. Additionally, sellers can view and respond to inquiries from potential buyers, which improves communication and increases the chances of successful property transactions.

The system also includes a powerful admin panel, which acts as the central control unit of the platform. The admin has full authority to manage users, verify accounts, and approve or reject property listings before they are published on the website. This helps ensure that only authentic and valid listings are displayed to users. The admin panel also supports content moderation, fraud detection, and system monitoring to maintain platform security and reliability. It allows administrators to track platform activity, manage reported listings, and ensure smooth operation of the entire system.

Another advanced deliverable is the AI-based recommendation system, which enhances user experience by suggesting relevant properties based on user behavior. This system analyzes user search history, clicked properties, saved listings, and location preferences to generate personalized recommendations. As a result, users can quickly discover properties that match their interests without manually searching, which improves engagement and user satisfaction.

The final deliverable is a secure central database system that stores all platform data in an organized and structured manner. This includes user profiles, property listings, images, inquiries, messages, and system logs. The database ensures data consistency, integrity, and security while supporting future scalability as the number of users and listings increases. It is designed to handle large volumes of data efficiently and provide fast access to information when required.

### 2.2.2 Development Requirements

The development of the Apni Jgah platform is based on modern web technologies to ensure performance, security, scalability, and ease of maintenance. The frontend of the system is developed using HTML, CSS, and JavaScript. HTML is responsible for structuring the web pages, CSS is used for designing and styling the user interface, and JavaScript adds interactivity and dynamic behavior to the website. Together, these technologies ensure that the platform provides a smooth, responsive, and user-friendly experience across all devices and screen sizes.

The backend of the system is developed using Django, a high-level Python web framework that follows the Model-View-Template (MVT) architecture. Django manages all server-side operations such as request handling, URL routing, business logic, authentication, and database communication. It also provides built-in security features that protect the system from common vulnerabilities like SQL injection, Cross-Site Request Forgery (CSRF), and Cross-Site Scripting (XSS). This makes the system highly secure and reliable for handling sensitive user and property data.

For database management, SQLite is used during the development phase due to its simplicity and ease of setup. However, for production-level deployment, PostgreSQL is recommended as it supports advanced features, better performance, and scalability for large datasets. The database stores all essential information including user accounts, property details, images, messages, and transaction-related data in a structured format.

Security and authentication are handled using Django's built-in authentication system. This system allows secure user registration, login, and session management. Passwords are stored in hashed form to ensure that sensitive user credentials are protected. Depending on system requirements, session-based authentication or token-based authentication (such as JWT) can be implemented, especially if APIs or mobile applications are integrated in the future.

If the system includes premium features such as featured property listings, advertisements, or priority placement, secure payment gateway integration can be added. Services such as JazzCash, Easypaisa, or bank payment APIs can be used to handle financial transactions securely and efficiently.

For deployment, the system can be hosted on cloud platforms such as AWS, Heroku, or other similar hosting services. Cloud deployment ensures high availability, scalability, and reliable performance even when user traffic increases. It also allows easy maintenance and updates without affecting system availability.

Additional development tools include Django REST Framework (DRF) for building APIs that can be used for mobile applications or future system expansion. Git and GitHub are used for version



control, enabling collaborative development among team members and maintaining proper code history and backup. This structured development approach ensures that the Apni Jgah system remains efficient, scalable, and maintainable in the long term.

## **2.3 Current System**

The current real estate system in Pakistan is outdated, fragmented, and lacks proper digital integration. Most property transactions are still handled through physical agents, local brokers, and informal advertising methods. Although some online platforms exist, they are not fully reliable, consistently updated, or properly standardized. In most cases, users depend heavily on agents to find suitable properties, which increases overall cost and reduces transparency in the process. Property information is often inconsistent, with missing details, outdated prices, and sometimes unverified ownership records. This creates confusion among users and reduces trust in the overall system. There is also no single centralized platform where users can easily compare multiple properties in one place, which forces buyers to manually check different sources and makes the entire process inefficient and time-consuming. Communication between parties is also indirect and mostly depends on intermediaries, which further slows down decision-making.

A large portion of real estate activity still relies on manual listings managed by agents and brokers. Properties are commonly advertised through physical boards, newspapers, WhatsApp groups, and social media posts. These listings are often incomplete, unverified, and inconsistent in format and detail. As a result, issues such as duplicate listings, fake advertisements, and outdated property information are very common. Users often waste time visiting irrelevant or unavailable properties, which leads to frustration and reduces confidence in the system. This lack of structured information makes it difficult for users to make quick and accurate decisions.

Another major issue is the absence of a centralized system that integrates all real estate services in one platform. Users are required to switch between different sources or depend on agents to collect complete information. There is no unified platform that offers search, filtering, communication, and property management together. This fragmented structure results in inefficiency, lack of transparency, and increased dependency on third parties. It also limits users' ability to compare properties effectively, which negatively impacts decision-making.

Communication between buyers and sellers in the current system is also slow and unorganized. Most interactions depend on phone calls, messages, or agents, which often leads to delays and misunderstandings. There is no real-time communication system or structured inquiry management. In addition, users do not receive automated notifications about new listings, price changes, or property availability. Because of this, many important opportunities are missed, and users cannot respond quickly to market changes.

Security and trust issues are also a major concern in the current real estate environment. Many property listings are not verified, and fraudulent advertisements are frequently observed. Users often hesitate to share personal information due to a lack of secure systems. There is also no proper

mechanism for verifying property ownership or ensuring safe communication between buyers and sellers. This weak security structure reduces user confidence and makes the system unreliable for serious transactions.

Another limitation of existing systems is the lack of intelligent or smart features. Users are required to manually search through large amounts of data without any assistance or recommendations. There are no AI-based suggestions, predictive pricing tools, or personalized search results available in most platforms. This makes the process time-consuming and less efficient, especially for users who are not familiar with the market.

In addition to these issues, real estate information in smaller cities and rural areas is even more limited. Most properties in these regions are not listed online, and users rely entirely on local agents for information. This creates a significant gap between urban and rural real estate markets. Due to this imbalance, users in remote areas face difficulty accessing updated property listings and fair market prices, which further highlights the need for a more centralized, transparent, and intelligent system.

# **Chapter 3**

## **REQUIREMENT ANALYSIS**

### 3. Requirement Analysis

The Requirement Analysis chapter provides a comprehensive and well-structured understanding of what the proposed Apni Jgah real estate platform must accomplish in order to operate effectively within the modern digital property market. This chapter acts as a critical foundation for the entire system development process, as it transforms user needs, expectations, and real-world challenges into clearly defined technical, functional, and system requirements. By identifying system behavior, user interactions, and the specific tasks the system must perform, it ensures that the development process remains focused, organized, and aligned with the intended objectives.

This chapter explains in detail how the Apni Jgah system is expected to function in a real-world environment. It outlines both functional requirements, which describe the core features and operations of the system such as property listing, searching, user registration, and communication, and non-functional requirements, which define system qualities like performance, security, reliability, scalability, and usability. Together, these requirements ensure that the system is not only functional but also efficient, secure, and user-friendly.

Furthermore, this chapter clearly defines the roles and responsibilities of all key users involved in the system, including buyers, sellers, agents, and administrators. Each user type interacts with the system in a unique way, and their actions are carefully analyzed to ensure that the platform meets their specific needs. Buyers are provided with tools to search and explore properties efficiently, sellers are given features to list and manage their properties, agents facilitate communication and transactions, and administrators oversee and control the overall system operations.

Using the Use Case Diagram as a reference, the system requirements are organized in a logical and structured manner. This diagram visually represents the interactions between users and the system, helping developers and stakeholders clearly understand how different components of the system work together. It also plays a key role in guiding the design, development, and testing phases by ensuring that all functionalities are properly defined and implemented.

Through this detailed analysis, the development team gains a clear understanding of the system's objectives, limitations, constraints, and expected performance levels. It helps in identifying potential challenges early in the development process and ensures that appropriate solutions are designed in advance. Most importantly, this chapter ensures that the final system effectively addresses major problems present in traditional real estate processes, such as manual property listings, lack of transparency, inefficient communication, and difficulties in property searching.

### 3.1 Use Case Diagram(s)

A use case diagram is a graphical representation that illustrates how different users interact with the system. It identifies the main actors and their interactions with system functionalities. Apni Jaga is a real estate platform designed to bring together property buyers, sellers, and administrators under one unified digital system. The use case diagram illustrates how three distinct actors the Buyer, the Seller, and the Admin interact with the system through a total of 16 defined use cases, all enclosed within the system boundary labeled "Apni Jaga."

The first thing to notice is that all three actors Buyer, Seller, and Admin share two fundamental use cases: Register Account and Login. This tells us that the platform uses a single unified authentication system where anyone wishing to use the platform, regardless of their role, must first create an account and then log in to access their features. This design choice ensures that every user on the platform is identifiable and accountable, which is especially important in a real estate context where transactions involve significant financial decisions.

The Buyer is the most feature-rich actor in the system, connected to ten different use cases, which reflects the fact that this platform is primarily designed around the buyer's experience. Once logged in, a Buyer can Search Properties, which allows them to look up available real estate listings using keywords, location names, or property types. To refine their results further, they can use the Filter Properties feature, which lets them narrow down listings based on criteria such as price range, property size, number of rooms, or locality. After finding a property of interest, the Buyer can View Property Details to access comprehensive information about that listing, including photographs, pricing, location on a map, available amenities, and the seller's contact information. If a Buyer finds a property they like but are not yet ready to make a decision, they can Add to Favorites to save that listing, and later revisit it through the View Favorite List feature, which aggregates all saved properties in one convenient place for easy comparison and review.

Beyond just browsing, the Buyer also has access to a ChatBot, which is an AI-powered conversational assistant that can answer questions about properties, guide users through the platform, or provide instant support without requiring human intervention. This feature significantly improves the user experience by making help available at any time. Another highly advanced feature available to the Buyer is AI Price Prediction, which leverages artificial intelligence to estimate the fair market value of a property based on various inputs such as location, size, nearby amenities, and historical pricing data. This empowers the Buyer to make more informed decisions and avoid overpaying for a property. Additionally, the Buyer can Read News, allowing them to stay updated on real estate market trends, new developments, policy changes, and investment opportunities published directly on the platform.

The Seller, on the other hand, has a more focused and specialized role within the system. Apart from the shared Register Account and Login use cases, the Seller has one exclusive but highly

comprehensive use case: Add, Edit, Delete and View Property Listing. This single use case encompasses the full lifecycle of a property listing. A Seller can add a brand-new listing with all relevant details and photographs, edit an existing listing if the price or details change, delete a listing once a property has been sold or is no longer available, and view all their active listings in one place. This streamlined design keeps the Seller's interaction simple and task-focused, ensuring they can manage their inventory efficiently without being overwhelmed by unnecessary features.

The Admin is the backbone of the platform's governance and management. Like the other actors, the Admin can Register Account and Login, giving them access to a powerful administrative dashboard. The Admin's exclusive use cases revolve entirely around platform control and oversight. Through Create, Update and Delete News, the Admin is responsible for all content published in the news section of the platform, ensuring that only accurate, relevant, and up-to-date real estate news reaches the users. The Admin also has the ability to Manage User Accounts, which means they can view the full list of registered users, review their activity, and take action when necessary. This is closely related to the Activate / Deactivate User use case, which allows the Admin to temporarily suspend a user's account for instance, if a seller is posting fraudulent listings or a buyer is engaging in suspicious behavior without permanently removing them from the system. If a violation is severe enough, the Admin can use the Delete User Account feature to permanently remove a user from the platform entirely. Finally, the Admin can Monitor Property Listing, which gives them oversight over all listings posted by sellers across the platform, allowing them to flag, review, or remove any listing that violates community guidelines or appears to be misleading or fraudulent.

Taken together, this use case diagram paints a clear picture of Apni Jaga as a well-structured, role-based real estate platform. The separation of responsibilities between Buyer, Seller, and Admin ensures that each type of user has access to exactly the features they need without unnecessary complexity. The inclusion of modern features like AI Price Prediction and a ChatBot shows that Apni Jaga is not just a traditional property listing website but a smart, technology-driven platform aimed at making real estate transactions more transparent, efficient, and accessible for everyone involved.



Figure 3.1:Apni Jgah Use Case Diagram

## **3.2 Detailed Use Case**

The detailed use cases explain in depth how each actor Buyer, Seller, and Admin interacts with the Apni Jaga system to accomplish specific goals. These use cases go beyond simple functionality by clearly describing the complete flow of actions, including what the user does, how the system responds, and how data is processed at each step. They provide a structured view of system behavior, making it easier to understand how the application operates in real-world situations.

Each use case typically begins with a trigger, such as a user selecting an option like login, searching for a property, or adding a listing. It then defines the preconditions, which are the conditions that must be satisfied before the action can take place (for example, a user must be registered before logging in). After that, the use case outlines the main flow of events, which includes all the steps performed by the actor and the corresponding responses generated by the system. In addition to the main flow, use cases may also include alternative or exception flows, which describe what happens if an error occurs (such as invalid login credentials or missing input data). Finally, each use case ends with a postcondition, indicating the final state of the system after the process is completed.

For the Buyer, detailed use cases describe activities such as creating an account, logging in, searching and filtering properties, viewing detailed property information, saving properties to a favorites list, and using advanced features like AI-based price prediction. These use cases show how the system helps buyers easily explore available properties, compare options, and make informed decisions. For the Seller, use cases focus on property management tasks. These include adding new property listings, updating existing information, uploading images, and deleting listings when necessary. The system ensures that sellers can efficiently manage their properties while maintaining accurate and up-to-date information for potential buyers. For the Admin, detailed use cases highlight system control and monitoring activities. Admins can manage user accounts, activate or deactivate users, remove accounts if needed, and oversee all property listings to ensure compliance and authenticity. They are also responsible for managing platform content, such as creating and updating news, which keeps users informed about market trends and updates. Overall, detailed use cases play a crucial role in system design and development. They ensure that all user interactions are clearly defined, reduce ambiguity in requirements, and help developers, designers, and stakeholders understand exactly how the system should behave. This leads to a more reliable, user-friendly, and well-structured real estate platform.

### **3.2.1 User Registration**

The User Registration use case describes the process through which a new user, either a buyer or a seller, creates an account in the Apni Jaga system. This is the first interaction a user has with the platform and is essential for accessing personalized features such as property search, favorites, and listing management. During this process, the user provides necessary details such as email,



username, and password. The system validates the input to ensure correctness, such as checking for a valid email format and ensuring that the username is unique. If all conditions are satisfied, the system securely stores the user's information in the database and confirms successful registration. This use case ensures that only authentic and unique users can access the system, thereby maintaining data integrity and security. It also forms the foundation for all subsequent user interactions within the platform.

**Table 3.1: User Registration**

|                  |                                                                                                                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-01                                                                                                                                                                                                          |
| Use Case Name    | Register Account                                                                                                                                                                                               |
| Actors           | Buyer, Seller                                                                                                                                                                                                  |
| Description      | Users can register by providing email, username, and password to create an account.                                                                                                                            |
| Trigger          | User clicks on 'Register' button.                                                                                                                                                                              |
| Preconditions    | User must have a valid email.                                                                                                                                                                                  |
| Postconditions   | User account created successfully.                                                                                                                                                                             |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. User opens registration page.</li> <li>2. Enters email, username, password.</li> <li>3. System validates and stores data.</li> <li>4. Confirmation shown.</li> </ol> |
| Alternative Flow | Invalid email or duplicate username → Show error.                                                                                                                                                              |
| Business Rules   | BR-1: Email must be unique.<br>BR-2: Password $\geq$ 8 characters.                                                                                                                                             |
| Assumptions      | User has internet and valid email.                                                                                                                                                                             |

### 3.2.2 Login and Authentication

The Login and Authentication use case explains how registered users securely access the Apni Jaga system. This process ensures that only authorized individuals can use the system's features based on their roles, such as buyer, seller, or admin. The user enters their credentials, which are verified against stored data in the database. If the credentials are correct and the account is active, the system grants access and redirects the user to the appropriate dashboard. In case of incorrect credentials or inactive accounts, the system displays an error message and denies access. This use case plays a critical role in maintaining system security, protecting user data, and ensuring that sensitive operations are performed only by authorized users.

**Table 3.2: Login and Authentication**

|                  |                                                                                 |
|------------------|---------------------------------------------------------------------------------|
| Use Case ID      | UC-02                                                                           |
| Use Case Name    | Login                                                                           |
| Actors           | Buyer, Seller, Admin                                                            |
| Description      | Authenticates users using their credentials.                                    |
| Trigger          | User clicks 'Login'.                                                            |
| Preconditions    | User must be registered.                                                        |
| Postconditions   | User logged in successfully.                                                    |
| Normal Flow      | 1. User enters credentials.<br>2. System verifies.<br>3. Redirect to dashboard. |
| Alternative Flow | Wrong credentials → Error message.                                              |
| Business Rules   | Only active accounts can log in.                                                |
| Assumptions      | Database connectivity available.                                                |

### 3.2.3 Search Properties

The Search Properties use case describes how buyers find suitable properties within the system. It allows users to enter keywords such as location, price range, or property type to retrieve relevant results. The system processes the search request by querying the database and returning matching property listings in a structured format. The system may also handle cases where no exact matches are found by suggesting similar properties. This use case ensures that buyers can easily navigate through a large number of listings and identify options that meet their needs.

**Table 3.3: Search Properties**

|                  |                                                                                 |
|------------------|---------------------------------------------------------------------------------|
| Use Case ID      | UC-03                                                                           |
| Use Case Name    | Search Properties                                                               |
| Actors           | Buyer                                                                           |
| Description      | Buyer searches for available properties.                                        |
| Trigger          | Buyer enters search keywords.                                                   |
| Preconditions    | User must be logged in.                                                         |
| Postconditions   | Search results displayed.                                                       |
| Normal Flow      | 1. Buyer enters search.<br>2. System fetches results.<br>3. Display properties. |
| Alternative Flow | No matches → Suggest similar properties.                                        |
| Business Rules   | Search results load within 3 seconds.                                           |
| Assumptions      | Database contains property data.                                                |

### 3.2.4 Add to Favorites

The Add to Favorites use case explains how buyers can save specific properties for future reference. When a buyer finds a property of interest, they can add it to their favorites list with a single action. The system records this selection and stores it in the user's profile. This feature helps users keep track of preferred properties without having to search again. It also enhances user convenience by allowing easy comparison of multiple properties at a later time. The system ensures that duplicate entries are avoided and provides feedback if a property is already added to the list.

**Table 3.4: Favorites**

|                  |                                                                                          |
|------------------|------------------------------------------------------------------------------------------|
| Use Case ID      | UC-04                                                                                    |
| Use Case Name    | Add to Favorites                                                                         |
| Actors           | Buyer                                                                                    |
| Description      | Allows buyers to mark properties as favorites.                                           |
| Trigger          | Buyer clicks 'Add to Favorites'.                                                         |
| Preconditions    | User logged in.                                                                          |
| Postconditions   | Property added to favorites.                                                             |
| Normal Flow      | 1. Buyer selects property.<br>2. Clicks 'Add to Favorites'.<br>3. System saves property. |
| Alternative Flow | Already favorited → Show message.                                                        |
| Assumptions      | User exists in database.                                                                 |

### 3.2.5 AI Price Prediction

The AI Price Prediction use case describes how the system utilizes artificial intelligence techniques to estimate the value of a property. When a user views property details, the system analyzes various factors such as location, size, historical data, and market trends. Based on this analysis, it generates an estimated price that helps users make informed decisions. This feature is particularly useful for buyers who want to evaluate whether a property is reasonably priced. In cases where sufficient data is not available, the system may notify the user that prediction is not possible. This use case adds advanced analytical capability to the platform, improving decision-making and user trust.

**Table 3.5: AI price prediction**

|                  |                                                  |
|------------------|--------------------------------------------------|
| Use Case ID      | UC-05                                            |
| Use Case Name    | AI Price Prediction                              |
| Actors           | Buyer                                            |
| Description      | AI predicts property value based on market data. |
| Trigger          | User views property details.                     |
| Preconditions    | Property data available.                         |
| Postconditions   | Predicted price displayed.                       |
| Normal Flow      | 1. AI analyzes data.<br>2. Show predicted price. |
| Alternative Flow | Missing data → 'Price unavailable'.              |
| Assumptions      | AI model trained and active.                     |

### 3.2.6 Filter Property

The Filter Property feature helps users narrow down search results by applying criteria like price, location, size, and type. It updates results based on selected filters to show only relevant properties. If no matches are found, the system notifies the user and may suggest alternatives.

**Table 3.6: Filter Property**

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| Use Case ID      | UC-06                                                                       |
| Use Case Name    | Filter Property                                                             |
| Actors           | Buyer                                                                       |
| Description      | Allows filtering of properties by location, price, size, type.              |
| Trigger          | Buyer applies filters.                                                      |
| Preconditions    | User logged in.                                                             |
| Postconditions   | Filtered list displayed.                                                    |
| Normal Flow      | 1. Buyer selects filters.<br>2. System applies filters.<br>3. Show results. |
| Alternative Flow | No results → Show message.                                                  |
| Business Rules   | AND logic is used for filters.                                              |
| Assumptions      | Property data has all fields.                                               |

### 3.2.7 View Property Details

The View Property Details use case explains how users access complete information about a selected property. When a property is selected from the list, the system retrieves detailed data, including images, price, location, description, and seller information. This use case is crucial as it provides users with all the necessary information required to evaluate a property. It ensures that data is presented clearly and accurately, helping users make informed decisions. If the property is no longer available, the system displays an appropriate message to inform the user.

**Table 3.7: View Property Details**

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| Use Case ID      | UC-07                                                                      |
| Use Case Name    | View Property Details                                                      |
| Actors           | Buyer, Seller                                                              |
| Description      | Displays detailed information about a selected property.                   |
| Trigger          | User clicks property listing.                                              |
| Preconditions    | Property exists in database.                                               |
| Postconditions   | Property details displayed.                                                |
| Normal Flow      | 1. User selects property.<br>2. System retrieves details.<br>3. Show info. |
| Alternative Flow | Property deleted → Show error.                                             |
| Assumptions      | Database active.                                                           |

### 3.2.8 View Favorite List

The View Favorite List use case describes how buyers can access all properties they have previously saved. This feature allows users to review, compare, and manage their selected properties in one place. The system retrieves the list of saved properties from the database and displays them in an organized manner. If the list is empty, the system informs the user accordingly. Additionally, users are provided with options to remove properties from the favorites list or navigate to the detailed property view for further information. The system ensures that the favorite list is updated in real time, reflecting any changes such as property availability or updates in details. This feature enhances user convenience by supporting quick decision-making and reducing the need for repeated searches.

**Table 3.8: View Favorites**

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Use Case ID      | UC-08                                                                 |
| Use Case Name    | View Favorite List                                                    |
| Actors           | Buyer                                                                 |
| Description      | Allows buyers to view properties they have favorited.                 |
| Trigger          | Buyer clicks 'Favorites'.                                             |
| Preconditions    | User logged in.                                                       |
| Postconditions   | Favorite list displayed.                                              |
| Normal Flow      | 1. Buyer clicks favorites.<br>2. System fetches and shows properties. |
| Alternative Flow | No favorites → Empty list message.                                    |
| Assumptions      | Favorites saved correctly.                                            |

### 3.2.9 Chatbot

The Chatbot use case describes how users interact with an intelligent virtual assistant integrated into the system. The chatbot uses natural language processing to understand user queries and provide relevant responses. Users can ask questions related to property search, pricing, or general platform usage. The system processes these queries and generates appropriate replies in real time. This feature improves user engagement and provides instant support without human intervention. In case of system limitations or errors, the chatbot informs the user and suggests trying again later.

**Table 3.9: Chatbot**

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| Use Case ID      | UC-09                                                                       |
| Use Case Name    | Chatbot                                                                     |
| Actors           | Buyer, Seller                                                               |
| Description      | Provides automated chat assistance to users.                                |
| Trigger          | User opens chatbot.                                                         |
| Preconditions    | AI system active.                                                           |
| Postconditions   | Chat session started.                                                       |
| Normal Flow      | 1. User opens chatbot. 2. AI prompts query.<br>3. User asks. 4. AI replies. |
| Alternative Flow | AI fails → 'Try again later'.                                               |
| Assumptions      | Internet available.                                                         |

### 3.2.10 Read News

The Read News use case explains how users access the latest updates and information related to the real estate market. The system retrieves news articles stored in the database and displays them in an organized format. This feature keeps users informed about market trends, new developments, and important announcements. The system may also display the publication date and category of each article to improve clarity and organization. This feature enhances user engagement by providing relevant and timely information in an easy-to-access format.

**Table 3.10: Read News**

|                  |                                                                        |
|------------------|------------------------------------------------------------------------|
| Use Case ID      | UC-10                                                                  |
| Use Case Name    | Read News                                                              |
| Actors           | Buyer, Seller                                                          |
| Description      | Displays latest real estate news updates.                              |
| Trigger          | User opens 'News' section.                                             |
| Preconditions    | News available in database.                                            |
| Postconditions   | News displayed.                                                        |
| Normal Flow      | 1. User clicks 'News'.<br>2. System fetches articles.<br>3. Show news. |
| Alternative Flow | No news → Show message.                                                |
| Assumptions      | Admin uploaded articles.                                               |

### 3.2.11 Manage News

The Manage News use case describes how administrators control the news content available on the platform. Admins can create new articles, update existing ones, or delete outdated information. The system ensures that only authorized users can perform these actions. This use case is important for maintaining accurate and up-to-date content, which helps users stay informed about the real estate market. Additionally, the system allows administrators to categorize news articles and manage publication dates for better organization. It also supports preview functionality, enabling admins to review content before publishing it to users. This feature ensures consistency, improves content quality, and enhances the overall reliability of information presented on the platform.

**Table 3.11: Update News**

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| Use Case ID      | UC-11                                                                      |
| Use Case Name    | Create, Update, and Delete News                                            |
| Actors           | Admin                                                                      |
| Description      | Admin can manage property-related news articles.                           |
| Trigger          | Admin opens 'Manage News'.                                                 |
| Preconditions    | Admin logged in.                                                           |
| Postconditions   | News updated successfully.                                                 |
| Normal Flow      | 1. Admin opens panel.<br>2. Adds/Edits/Deletes news. 3. System updates DB. |
| Alternative Flow | Invalid data → Show error.                                                 |
| Assumptions      | Admin privileges granted.                                                  |

### 3.2.12 Manage User Account

The Manage User Account use case explains how users can update their personal information such as profile details, contact information, and passwords. It also allows administrators to oversee user accounts when necessary. The system validates all changes before saving them to ensure data accuracy. This use case ensures that user information remains current and secure.

**Table 3.12: Manage User Account**

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| Use Case ID      | UC-12                                                                   |
| Use Case Name    | Manage User Account                                                     |
| Actors           | Admin, Seller                                                           |
| Description      | Manage or update user profile, contact, and password.                   |
| Trigger          | User accesses 'Manage Account'.                                         |
| Preconditions    | User logged in.                                                         |
| Postconditions   | Profile updated successfully.                                           |
| Normal Flow      | 1. User opens settings.<br>2. Updates info.<br>3. System saves changes. |
| Alternative Flow | Missing fields → Show error.                                            |
| Assumptions      | Database updates allowed.                                               |



### 3.2.13 Manage Property Listing

The Manage Property Listing use case describes how sellers and administrators handle property listings within the system. Sellers can add new properties, update details, upload images, or delete listings that are no longer available. The system ensures that all information is validated before being stored. This use case is essential for maintaining accurate and up-to-date property data, which directly impacts user trust and system reliability. Additionally, the system allows users to view a list of all their active and inactive property listings in one place for better management. It also provides feedback messages to confirm successful actions such as updates or deletions. This feature improves efficiency by enabling quick modifications and ensures that property information remains consistent and reliable for all users.

**Table 3.13: Update Property**

|                  |                                                       |
|------------------|-------------------------------------------------------|
| Use Case ID      | UC-13                                                 |
| Use Case Name    | Add/Edit/Delete/View Property Listing                 |
| Actors           | Seller, Admin                                         |
| Description      | Allows sellers/admin to manage property listings.     |
| Trigger          | Seller selects 'Manage Property'.                     |
| Preconditions    | Seller logged in.                                     |
| Postconditions   | Property list updated.                                |
| Normal Flow      | 1. Seller adds or edits listing.<br>2. Saves changes. |
| Alternative Flow | Invalid data → Error message.                         |
| Assumptions      | Seller has permissions.                               |

### 3.2.14 Activate/Deactivate User

The Activate/Deactivate User use case explains how administrators control access to the system by enabling or disabling user accounts. This feature is useful for handling suspicious activities, policy violations, or inactive users. The system updates the account status accordingly and restricts access when necessary. This use case enhances system security and ensures proper user management.

**Table 3.14: Activate/Deactivate User**

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Use Case ID      | UC-14                                                                 |
| Use Case Name    | Activate/Deactivate User                                              |
| Actors           | Admin                                                                 |
| Description      | Enables or disables user accounts.                                    |
| Trigger          | Admin selects 'Activate/Deactivate'.                                  |
| Preconditions    | Admin logged in.                                                      |
| Postconditions   | User account status changed.                                          |
| Normal Flow      | 1. Admin opens user list.<br>2. Toggles status.<br>3. System updates. |
| Alternative Flow | Invalid user → Error.                                                 |
| Assumptions      | Admin privileges available.                                           |

### 3.2.15 Delete User Account

The Delete User Account use case describes how administrators permanently remove a user from the system. This action is typically performed when a user violates policies or requests account deletion. The system ensures that the deletion is confirmed before removing all associated data. This use case helps maintain system integrity and prevents misuse.

**Table 3.15: Delete User Account**

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Use Case ID      | UC-15                                                                 |
| Use Case Name    | Delete User Account                                                   |
| Actors           | Admin                                                                 |
| Description      | Permanently deletes user account.                                     |
| Trigger          | Admin chooses 'Delete User'.                                          |
| Preconditions    | User exists in database.                                              |
| Postconditions   | User deleted successfully.                                            |
| Normal Flow      | 1. Admin selects user.<br>2. Confirms deletion.<br>3. System deletes. |
| Alternative Flow | User not found → Error message.                                       |
| Assumptions      | Deletion irreversible.                                                |

### 3.2.16 Monitor Property Listing

The Monitor Property Listing use case explains how administrators oversee all property listings to ensure they meet system standards and guidelines. Admins review listings, verify information, and take necessary actions such as editing or removing inappropriate content. This use case is important for maintaining the quality, reliability, and authenticity of the platform.

**Table 3.16: Monitor Property Listing**

|                  |                                                                           |
|------------------|---------------------------------------------------------------------------|
| Use Case ID      | UC-16                                                                     |
| Use Case Name    | Monitor Property Listing                                                  |
| Actors           | Admin                                                                     |
| Description      | Allows admin to oversee property listings.                                |
| Trigger          | Admin opens 'Monitor Property'.                                           |
| Preconditions    | Admin logged in.                                                          |
| Postconditions   | Property list reviewed.                                                   |
| Normal Flow      | 1. Admin views listings and Checks details.<br>3. Takes action if needed. |
| Alternative Flow | No listings → Show message.                                               |
| Assumptions      | System logs are maintained.                                               |

## 3.3 Functional Requirements

The Real Estate Website must provide the following functional requirements to support buyers, sellers, agents, and administrators. These requirements define what the system should do and how users will interact with it to search, list, manage, and compare properties effectively. The system should allow users to create and manage accounts with secure authentication, enabling different roles such as buyer, seller, agent, and administrator. Buyers should be able to search for properties using multiple filters such as location, price range, property type, size, and availability. The system should also support advanced search options to help users quickly find relevant properties according to their preferences. Sellers and agents must be able to add new property listings, update existing property details, and remove outdated listings. Each property listing should include essential information such as images, description, price, location, number of rooms, and other relevant features. The system should ensure that all listings are properly validated before being published.

Users should be able to view detailed property pages, including image galleries and full specifications. A comparison feature should allow buyers to compare multiple properties side by

side to make better decisions. Additionally, users should be able to save favorite properties to a Wishlist for future reference. The system should also provide an inquiry or contact feature so that buyers can directly communicate with sellers or agents regarding property details. Administrators should have full control over the platform, including managing users, approving listings, and monitoring system activity to ensure data accuracy and platform security.

Overall, these functional requirements ensure that the system is user-friendly, efficient, and capable of handling all core operations of a real estate platform smoothly.

### User Authentication

This describes the functionality for user registration and login, ensuring that users can securely access the system using valid credentials. It includes password encryption, authentication mechanisms, and role-based access control to differentiate between user types such as buyers, sellers, and administrators. This feature protects user data, prevents unauthorized access, and enables personalized system interaction.

**Table 3.17: User Authentication**

|                       |                                                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-1</b>                                                                                                                      |
| <b>Title</b>          | User Authentication and Registration                                                                                             |
| <b>Requirement ID</b> | FR-1                                                                                                                             |
| <b>Requirement</b>    | The system shall allow users to register and log in using valid credentials, ensuring password encryption and role-based access. |
| <b>Source</b>         | Client Requirement                                                                                                               |
| <b>Rationale</b>      | Secure access control for all user roles.                                                                                        |
| <b>Business Rule</b>  | Only verified users can log in.                                                                                                  |
| <b>Dependencies</b>   | Database connection, encryption module                                                                                           |
| <b>Priority</b>       | High                                                                                                                             |

### Property Listing Management

This defines the feature that allows users to find properties quickly based on specific criteria, improving efficiency and user experience.

**Table 3.18: Property Listing Management**

|                       |                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-2</b>                                                                                                                  |
| <b>Title</b>          | Property Listing Management                                                                                                  |
| <b>Requirement ID</b> | FR-2                                                                                                                         |
| <b>Requirement</b>    | The system shall allow sellers to create, update, and remove property listings with full details stored in the SQL database. |
| <b>Source</b>         | System Requirement                                                                                                           |
| <b>Rationale</b>      | Enables sellers to manage their property data efficiently.                                                                   |
| <b>Business Rule</b>  | Only logged-in sellers can modify their listings.                                                                            |
| <b>Dependencies</b>   | Database module                                                                                                              |
| <b>Priority</b>       | High                                                                                                                         |

#### **Property Search and Filtering**

This defines the feature that allows users to search and filter properties based on various criteria such as location, price range, and property type. It improves user experience by providing fast, accurate, and relevant results. This functionality reduces search time and helps users easily find properties that match their preferences.

**Table 3.19: Property Search and Filtering**

|                       |                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-3</b>                                                                                                         |
| <b>Title</b>          | Property Search and Filtering                                                                                       |
| <b>Requirement ID</b> | FR-3                                                                                                                |
| <b>Requirement</b>    | The system shall provide search and filter options for buyers to find properties quickly based on defined criteria. |
| <b>Source</b>         | Functional Specification                                                                                            |
| <b>Rationale</b>      | Improves user experience and search accuracy.                                                                       |
| <b>Business Rule</b>  | Search must respond within 3 seconds for 95% of queries.                                                            |
| <b>Dependencies</b>   | Database query optimization                                                                                         |
| <b>Priority</b>       | High                                                                                                                |

## AI-Based Price Prediction

This describes the feature that uses AI techniques to estimate property prices based on factors such as location, size, and type. It provides users with useful insights into property value, helping them make informed decisions.

**Table 3.20: AI-Based Price Prediction**

|                       |                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-4</b>                                                                                             |
| <b>Title</b>          | AI-Based Price Prediction                                                                               |
| <b>Requirement ID</b> | FR-4                                                                                                    |
| <b>Requirement</b>    | The system shall predict property values using AI algorithms and display results with property details. |
| <b>Source</b>         | Supervisor Suggestion                                                                                   |
| <b>Rationale</b>      | Provides smart insights for buyers and sellers.                                                         |
| <b>Business Rule</b>  | Prediction results must generate within 5 seconds.                                                      |
| <b>Dependencies</b>   | AI model, data integration module                                                                       |
| <b>Priority</b>       | Medium                                                                                                  |

## Favorites Management

This explains how users can save properties to a favorites (Wishlist) list for future reference. It allows users to add, remove, and view saved properties easily. This feature improves personalization and helps users compare different properties without searching again.

**Table 3.21: Favorites Management**

|                       |                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-5</b>                                                                                                   |
| <b>Title</b>          | Favorites Management                                                                                          |
| <b>Requirement ID</b> | FR-5                                                                                                          |
| <b>Requirement</b>    | The system shall allow users to add or remove properties from their favorites list and view saved properties. |
| <b>Source</b>         | Client Requirement                                                                                            |
| <b>Rationale</b>      | Enhances personalized experience for buyers.                                                                  |
| <b>Business Rule</b>  | Only logged-in users can use favorites.                                                                       |
| <b>Dependencies</b>   | User profile module                                                                                           |
| <b>Priority</b>       | Medium                                                                                                        |

## AI Chatbot Support

This presents the chatbot functionality that provides instant assistance to users. It helps answer common questions, guide users through the system, and improve overall interaction. This feature reduces the need for manual support and enhances user experience by offering quick responses.

**Table 3.22: AI Chatbot Support**

|                       |                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-6</b>                                                                                       |
| <b>Title</b>          | AI Chatbot Support                                                                                |
| <b>Requirement ID</b> | FR-6                                                                                              |
| <b>Requirement</b>    | The system shall provide an AI chatbot to assist users with property queries and navigation help. |
| <b>Source</b>         | Functional Design                                                                                 |
| <b>Rationale</b>      | Provides instant user assistance and reduces admin workload.                                      |
| <b>Business Rule</b>  | The chatbot should respond within 2 seconds.                                                      |
| <b>Dependencies</b>   | AI module, Internet connection                                                                    |
| <b>Priority</b>       | Medium                                                                                            |

## Blogs and News Section

This describes the feature that allows administrators to post blogs and news related to real estate. Users can read and interact with this content, which helps them stay informed about market trends and useful information. This feature adds value to the platform by providing educational content.

**Table 3.23: Blogs and News Section**

|                       |                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-7</b>                                                                                          |
| <b>Title</b>          | Blogs and News Section                                                                               |
| <b>Requirement ID</b> | FR-7                                                                                                 |
| <b>Requirement</b>    | The system shall allow admins to manage blogs and news posts, and users to read and comment on them. |
| <b>Source</b>         | Client Suggestion                                                                                    |
| <b>Rationale</b>      | Keeps users informed and engaged.                                                                    |
| <b>Business Rule</b>  | Only approved users can comment.                                                                     |
| <b>Dependencies</b>   | Database, admin module                                                                               |
| <b>Priority</b>       | Low                                                                                                  |

## Admin Dashboard

This explains the admin dashboard, which provides complete control over the system. Administrators can manage users, monitor property listings, and oversee system activities. It ensures smooth operation, proper management, and effective monitoring of the platform.

**Table 3.24: Admin Dashboard**

|                       |                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>     | <b>FR-8</b>                                                                                                  |
| <b>Title</b>          | Admin Dashboard                                                                                              |
| <b>Requirement ID</b> | FR-8                                                                                                         |
| <b>Requirement</b>    | The system shall provide an admin dashboard for managing users, properties, and monitoring system analytics. |
| <b>Source</b>         | Project Requirement                                                                                          |
| <b>Rationale</b>      | Ensures smooth control and monitoring of platform operations.                                                |
| <b>Business Rule</b>  | Only admin role can access dashboard.                                                                        |
| <b>Dependencies</b>   | Authentication module                                                                                        |
| <b>Priority</b>       | High                                                                                                         |

## 3.4 Non-Functional Requirements

### Usability

The system shall provide a highly user-friendly and intuitive interface that can be easily understood by users with minimal or no technical background. The design will focus on simplicity, clarity, and ease of navigation so that users can perform tasks efficiently without requiring extensive training. The interface shall be responsive and fully compatible with different screen sizes, ensuring smooth accessibility across desktops, tablets, and smartphones.

Clear and well-organized navigation menus shall be provided for different user roles such as buyers, sellers, agents, and administrators, allowing each user to access relevant features. The system shall also support multi-language functionality to increase accessibility for a diverse user base. Additionally, proper feedback mechanisms such as error messages, alerts, and tooltips shall guide users in case of incorrect input, helping them quickly correct mistakes and continue their tasks without confusion.



## **Performance**

The system shall be designed to deliver fast and efficient performance under normal and peak usage conditions. It shall be capable of handling at least 200 concurrent users without noticeable degradation in speed or responsiveness. Property search operations shall be optimized so that at least 95% of search results are displayed within seconds, ensuring a smooth browsing experience.

## **Security**

The system shall implement strong security measures to protect user data and ensure safe transactions. Sensitive information such as passwords shall be encrypted, and personal data shall be securely stored to prevent unauthorized access. Role-based access control shall be applied, ensuring that only authorized users (such as administrators) can access sensitive features or perform critical operations.

The system shall include protection mechanisms against common threats such as unauthorized logins, data breaches, and malicious activities. Secure authentication methods, input validation, and session management techniques shall be implemented to maintain data privacy and integrity. Regular security updates and monitoring shall also be performed to keep the system protected against emerging threats.

## **Reliability**

The system shall ensure consistent and dependable operation under various conditions. It shall function correctly with minimal errors and provide stable performance over time. In the event of system failures or unexpected issues, the system shall be capable of recovering quickly without significant disruption to users.

Data backup mechanisms shall be implemented to regularly store copies of important information, ensuring that no critical data is lost. The system shall also include error-handling procedures to detect, report, and resolve issues efficiently, maintaining overall system stability.

## **Maintainability**

The system shall be designed with a modular and well-structured architecture to simplify maintenance and future enhancements. The codebase shall follow standard coding practices and be properly documented, making it easier for developers to understand, modify, and extend the system.

This approach will allow developers to fix bugs, update existing features, or add new functionalities without affecting other parts of the system. Regular updates and improvements shall

be easily implementable, ensuring that the system remains up-to-date with changing user requirements and technological advancements.

### **Portability**

The system shall be platform-independent and accessible through all major web browsers such as Chrome, Microsoft Edge, and Firefox. It shall function consistently across different operating systems and devices without requiring special configurations.

Users shall be able to access the system from any location using devices with an internet connection, ensuring flexibility and convenience. This portability ensures that the platform can reach a wider audience without technical limitations.

### **Scalability**

The system shall be capable of handling growth in terms of users, property listings, and system operations without compromising performance. The architecture shall support horizontal and vertical scaling, allowing resources to be increased as demand grows.

It shall also be designed in a way that supports the integration of future features such as advanced analytics, payment systems, or third-party services. This ensures that the platform remains sustainable and adaptable in the long term.

### **Availability**

The system shall be available to users **24/7**, ensuring continuous access to services at all times. Downtime shall be minimized, and any maintenance activities shall be scheduled during off-peak hours to reduce disruption to users.

The system shall include monitoring tools to detect and resolve issues proactively, ensuring smooth and uninterrupted operation. High availability mechanisms shall be implemented to maintain consistent service delivery.

### **Data Integrity**

The system shall ensure that all stored data remains accurate, consistent, and reliable throughout its lifecycle. Proper validation rules shall be applied during data entry to prevent incorrect or incomplete information from being stored.

# **Chapter 4**

## **DESIGN AND ARCHITECTURE**

## 4. Design and Architecture

This chapter describes the overall design of the Apni Jgah – AI Based Real Estate Price Prediction & Property Finder system. It explains how different components of the system are structured and how they interact with each other. The design ensures that the system is scalable, efficient, and easy to maintain.

### 4.1 System Architecture

**Presentation Layer:** The Presentation Layer is responsible for the user interface and user experience. It is developed using HTML, CSS, and JavaScript, providing a responsive and interactive design.

Key responsibilities include:

- Displaying property listings, dashboards, and search results
- Handling form validation (e.g., login, registration, property forms)
- Managing UI state and session handling
- Providing features like filters, favorites, and chatbot interface
- Ensuring responsiveness across devices

This layer communicates with the backend through APIs, sending user requests and displaying the responses received.

**Application Layer:** The Application Layer is the core part of the system where all business logic is implemented. It is built using the Django Framework and is responsible for processing user requests, applying system rules, and generating appropriate responses. This layer includes several modules such as authentication for handling login and registration, property management for managing listings, search and filter functionality for retrieving relevant properties, and an AI prediction engine for estimating property prices. It also includes a chatbot module that uses natural language processing to assist users, as well as a content management module for handling blogs and news. The admin panel is also part of this layer, allowing administrators to manage users, monitor listings, and control system activities. All these components interact through REST APIs, ensuring modularity and flexibility.

**Data Layer:** The Data Layer is responsible for storing and managing all the data used in the system. The application uses an SQLite database to store user information, property listings, images, favorite properties, blog content, and system logs. It also stores datasets used for training AI models, including historical property prices and market trends. The database ensures data integrity by maintaining relationships and constraints between different data entities. It supports efficient data retrieval for search, filtering, and analysis purposes. The connection between the application layer and data layer ensures that data is consistently stored and retrieved as needed.

**Integration Layer:** The Integration Layer ensures communication between the system and external services while maintaining security and reliability. It uses REST APIs for communication between frontend and backend, along with JWT authentication for secure access control. HTTPS and SSL encryption are used to protect data during transmission. This layer also manages error handling, request validation, and API rate limiting. Additionally, it integrates external services such as email gateways for sending notifications, SMS gateways for communication and verification, and mapping APIs for locationbased property features.

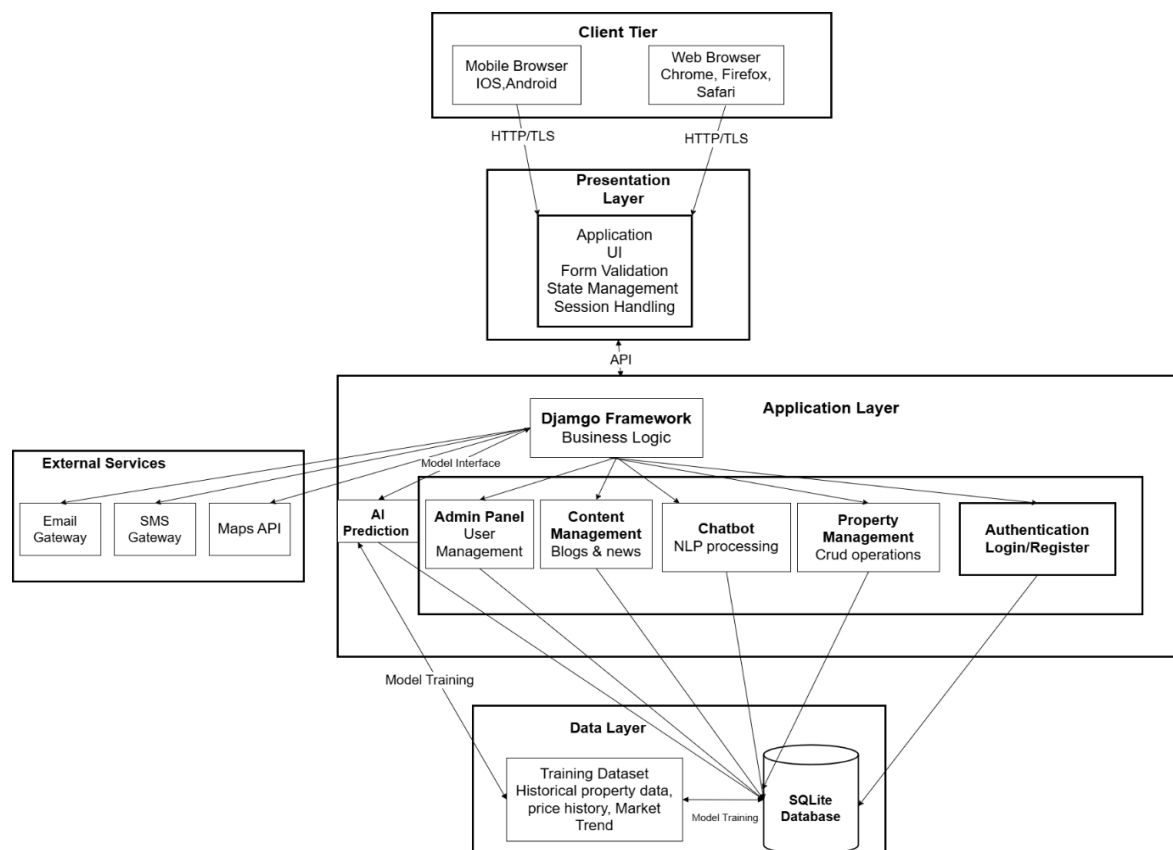


Figure 4.1: Three-Tier Architecture Diagram for Real Estate Platform

The diagram illustrates how these layers interact with each other in a structured manner. At the top, the client tier shows users accessing the system through mobile and web browsers. These requests are sent to the presentation layer using HTTP or HTTPS. The presentation layer then communicates with the application layer through APIs. Within the application layer, the Django framework acts as the central unit that connects different modules such as authentication, property management, chatbot, AI prediction, and admin controls. The diagram also shows external services connected to the application layer, indicating integration with email, SMS, and map services. At the bottom, the data layer contains the SQLite database and training datasets, which are used by the application layer for storing data and performing AI model training. The arrows in the diagram represent the flow of data and communication between components.

Overall, the architecture demonstrates a clear separation of concerns, where each layer has a specific responsibility. This design ensures that the system remains scalable, secure, efficient, and easy to maintain, while also supporting advanced features like AI based price prediction and chatbot interaction.

## **4.2 Data Representation**

The Data Representation section explains how information is structured, stored, and managed within the Apni Jgah real estate system. It covers the database design approach, entity relationships, data flow between modules, and the organization and security of stored data. This ensures that all system components can efficiently access and process property related information while maintaining accuracy, consistency, and data security.

### **4.2.1 Database Design**

Apni Jgah uses a relational database (SQLite) due to its simplicity, lightweight nature, and easy integration with web applications like Django.

The database is organized into tables, where each table represents a key entity such as Users, Properties, Agents, Bookings, and Payments. Each table contains structured fields (columns) with defined data types to ensure consistent data storage.

SQLite supports ACID properties, ensuring reliable transactions for operations such as property bookings and payment processing. This design allows efficient querying using SQL, maintains data integrity through constraints, and ensures smooth performance for managing property listings, user interactions, and real time updates.

### **4.2.2 Entity-Relationship Representation**

The system follows a relational database model in which entities are represented as tables and relationships are maintained through primary and foreign keys. The user table stores information about buyers, sellers, and administrators, while the property table contains details such as property title, price, location, and description. The agent table stores professional information about real estate agents.

Relationships are defined clearly within the system. A single user can list multiple properties, creating a one-to-many relationship. Each property can have multiple bookings, while each booking is associated with one specific user and one property. Payment records are linked to bookings to track financial transactions. An entity-relationship diagram is used to visually represent how these tables are connected, ensuring clarity in data organization and enabling efficient access and management of information.

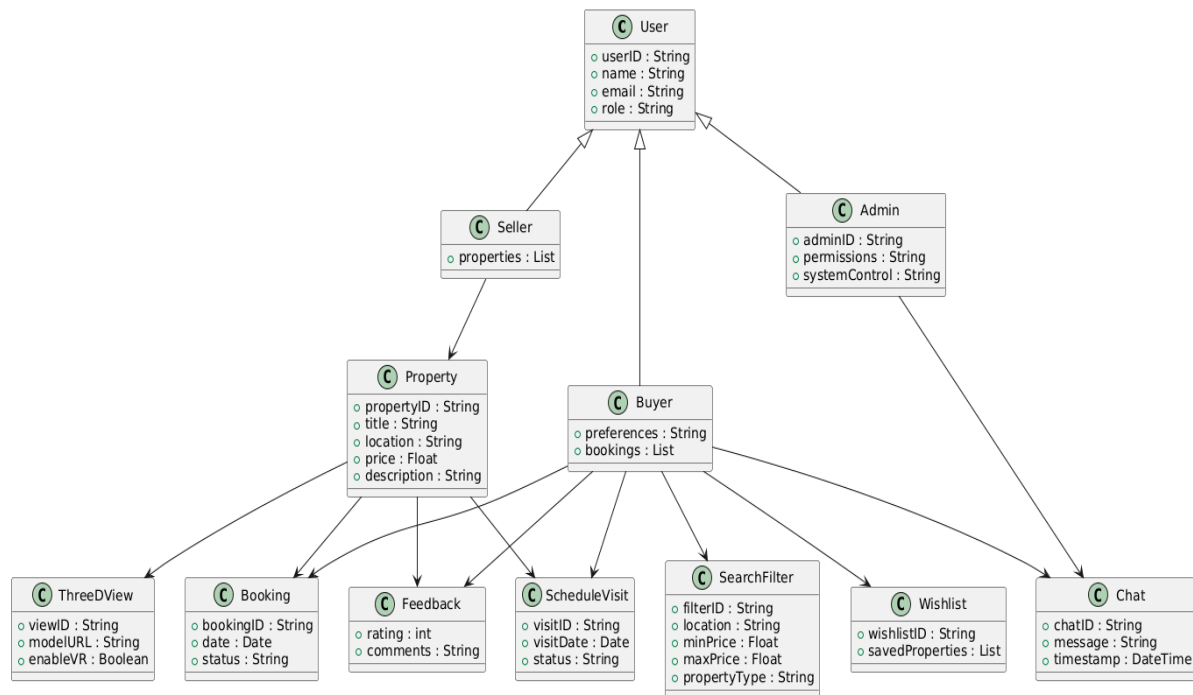


Figure 4.2: Entity-Relationship Diagram of Apni Jgah System

Figure 4.3 presents the complete relational database structure of the Real Estate (Apni Jgah) system implemented in SQLite. It illustrates how different tables are organized and connected to manage system data efficiently. The database follows a relational model where each table represents a core entity of the system such as User, Buyer, Seller, Admin, Property, Booking, Feedback, Wishlist, Chat, SearchFilter, ScheduleVisit, and ThreeDView. Each table contains a primary key (PK) that uniquely identifies each record, while foreign keys (FK) are used to establish relationships between tables. The User table acts as the main parent entity, and it is specialized into Buyer, Seller, and Admin tables through userID as a foreign key. The Seller table is associated with the Property table, allowing each seller to manage multiple properties. The Property table is central to the system and is linked with Booking, Feedback, Wishlist, ScheduleVisit, and ThreeDView tables to support different functionalities of the platform.

The Buyer table is connected with Booking, Feedback, Wishlist, SearchFilter, Chat, and ScheduleVisit tables, enabling buyers to interact with properties and sellers effectively. The Chat table facilitates communication between Buyer and Admin, while the SearchFilter table allows buyers to filter properties based on location, price range, and property type. The ScheduleVisit table manages property visit appointments between buyers and properties, ensuring proper scheduling functionality. The ThreeDView table stores 3D model data for properties, enabling interactive visualization features in the system. Overall, this relational structure ensures data consistency, reduces redundancy, and maintains proper relationships between all entities of the Real Estate system.

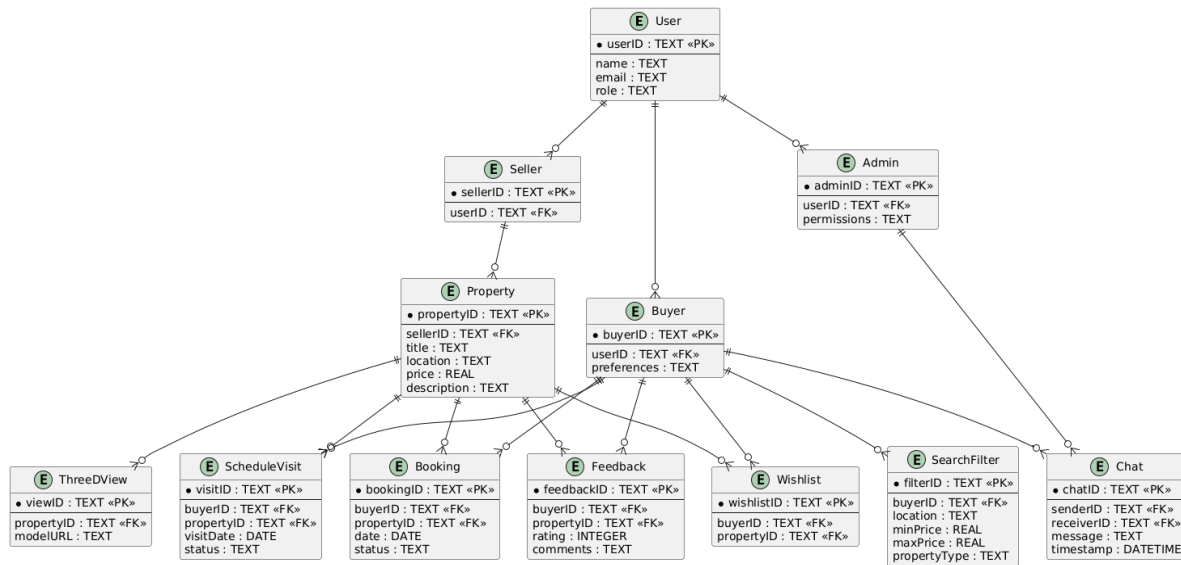


Figure 4.3 :Detailed Table Relationship Structure in SQLite

### 4.2.3 Data Flow

Data in the Apni Jgah system flows through a structured process that begins at the user interface and ends in secure database storage. When a user adds a property, the information is entered through the interface, validated in the business logic layer, and then stored in the SQLite database. Similarly, when a user searches for properties or books a visit, the request is processed by the system and the resulting data is stored or retrieved accordingly. This structured data flow ensures smooth coordination between different modules such as property listings, bookings, user management, and payments.

### 4.2.4 Entity Descriptions and Constraints

Each entity in the database performs a specific function within the system. The user table stores personal and authentication information, while the property table maintains details about listings, including price, location, and descriptions. The agent table contains professional details of agents associated with property listings. The booking table records interactions between users and properties, including booking dates and status. The payment table stores transaction details such as amount, date, and payment status.

The system enforces constraints to maintain data integrity. Each table includes a primary key for unique identification, while foreign keys are used to establish relationships between tables. Required fields are enforced using NOT NULL constraints, and unique fields such as email addresses prevent duplication. Data validation is handled at the application level as well as within the database. Security is maintained through authentication mechanisms, controlled access, and proper handling of sensitive user and transaction data.



### 4.3 Process Flow Representation

Figure 4.4 presents the overall workflow of the system, demonstrating how different types of users Buyer, Seller, and Admin interact with the platform and how their actions are processed. The flow begins with the **User Access System**, where a user enters the platform. The system then identifies the user type through a decision point labeled “Identify User Type,” which directs the flow into three separate paths based on whether the user is a Buyer, Seller, or Admin.

In the Buyer flow, the process starts with browsing or searching for properties. The system retrieves filtered results according to the buyer’s preferences, and the buyer can view detailed information about selected properties. After viewing property details, the buyer reaches a decision stage where different actions can be performed. The buyer may choose to open the blog section, where articles are retrieved and displayed as blog content. Alternatively, the buyer can send property data to an AI engine to receive a predicted price and then continue browsing. Another option allows the buyer to add a property to favorites, save it, and proceed with further browsing. The buyer may also choose to exit the system at any stage. All these activities ultimately lead to the completion of the buyer’s session.

In the Seller flow, the system provides property management functionality. After entering the system, the seller selects an operation, which can include creating, updating, or deleting a property listing. When creating a listing, the seller enters property details, which are then validated and saved. In the case of updating a listing, the seller modifies existing data and saves the updated information. If the seller chooses to delete a listing, the system removes it from the database. Each of these operations concludes with the seller ending their session.

In the Admin flow, the process involves managing the overall system. The admin accesses the admin panel and is responsible for managing users, properties, and blog content. The admin can approve or remove content and ensure that the database is updated accordingly. After completing these administrative tasks, the admin ends the session.

All three user flows eventually converge at the system’s termination point, indicating the end of interaction. This flowchart clearly represents the structured operation of the system, highlighting how different user roles are handled efficiently and how their respective actions are processed within the platform.

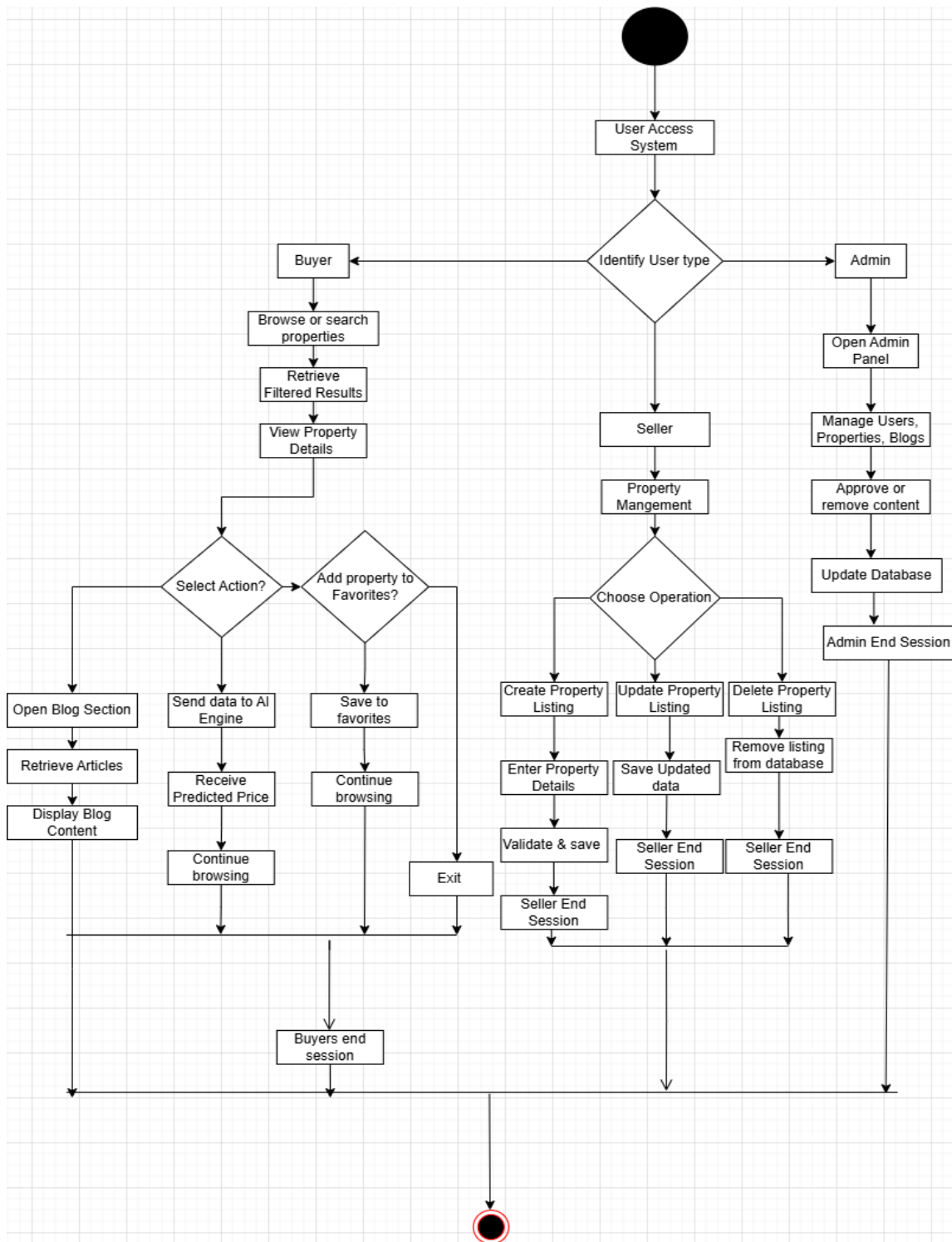


Figure 4.4: Process Flow Diagram for Apni Jgah

## 4.4 Design Models

This section presents the behavioral design models of Apni Jgah through a comprehensive set of Sequence Diagrams. These models detail the dynamic interactions between system actors (Buyer, Seller, Admin) and internal system components (Web Interface, Backend Services, Database, AI Engine, Chatbot) for each major use case. Together, they provide a complete picture of how the system responds to user actions at runtime.

### 4.4.1 Sequence Diagram

#### Buyer Sequence Diagram

Figure 4.5 illustrate how buyers, sellers, and admins interact with Apni Jgah. Buyers authenticate, search properties with AI based price predictions, and save favorites. This diagram represents the working flow of a real estate web system with an integrated AI-based price prediction feature. It shows how a buyer interacts with the system through the web interface and how different backend components work together to process requests.

The process starts when the buyer accesses the website. The web interface first communicates with the authentication controller to verify the user session. The authentication controller checks the session details from the database. If the session is valid, the system confirms that the user is authenticated and allows further actions.

After successful login, the buyer can search for properties by applying filters such as location, price range, and property type. This search request is sent from the web interface to the property controller. The property controller queries the database using the provided filters and retrieves matching properties. These results are sent back as a property list and displayed on the web interface for the user to view.

When the buyer selects a specific property, a request is sent to get detailed information about that property using its ID. The property controller fetches complete data from the database and returns it to the web interface. The system then displays full property details to the buyer.

The diagram also includes an AI price prediction feature. When the buyer requests a predicted price, the property data is sent to the AI price prediction service. This service processes the data by performing feature analysis and running a machine learning model. After processing, it generates a predicted price along with a confidence score. This result is then returned to the web interface and shown to the user.

Another important part of the system is the add to favorites functionality. When the buyer adds a property to favorites, the web interface sends a request to the property controller. The system first checks in the database whether the property already exists in the user's favorites list. If it does not exist, a new record is inserted and saved successfully, and the system confirms that the property has been added to favorites. If the property already exists, the system simply notifies the user that it is already in the favorites list, avoiding duplicate entries.

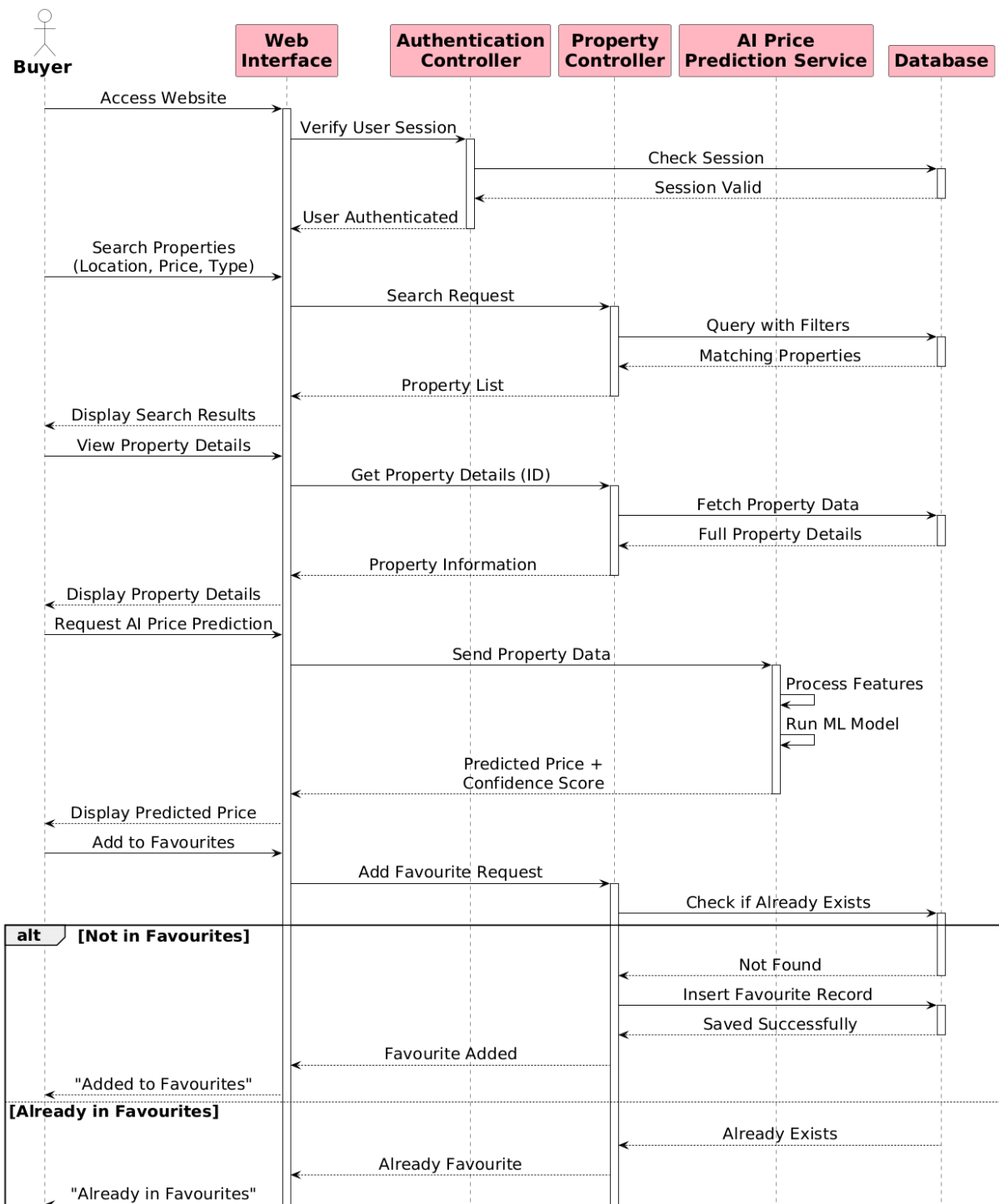


Figure 4.5: Buyer Property Search and Operations Sequence Diagram

## Seller Sequence Diagram

Figure 4.6 details the seller's interaction flow for creating, updating, and deleting property listings, illustrating the collaboration between the user interface and backend services. This shows how a Seller interacts with the system to manage property listings in a real estate web application. The interaction is divided into three main processes: creating, updating, and deleting property listings, and it shows how different system components communicate step by step.

In the first part, when the seller wants to create a property listing, they navigate to the “Create Listing” option through the web interface. The system first checks the seller’s role through the authentication controller to ensure the user is authorized. Once verified, the listing form is displayed where the seller enters property details such as title, price, location, and images. After submission, the data is sent to the property controller, which validates the input. If validation is successful, the system inserts the property record into the database, generates a property ID, and sets its status to “Pending” for admin approval. The email service then notifies the admin about the new listing. If validation fails, error messages are shown back to the seller instead of saving the data.

In the second part, the update property listing process begins when the seller navigates to “My Listings.” The system fetches all properties linked to that seller ID from the database and displays them. The seller selects a property to edit, and the system retrieves full property details and shows them in an edit form. After the seller modifies the information and submits changes, the property controller validates the updated data. If valid, the database record is updated successfully and confirmation is sent back to the seller.

In the third part, the delete property listing process starts when the seller selects a property to remove. A confirmation dialog appears to prevent accidental deletion. Once confirmed, a delete request is sent to the property controller, which verifies ownership of the property. After verification, the system updates the status to “Deleted” in the database instead of immediately removing the record. Once deletion is completed, the seller is notified that the property has been successfully removed.

Overall, the diagram illustrates a full workflow of property management with proper authentication, validation, database operations, and notification handling, ensuring secure and structured interaction between the seller and the system.

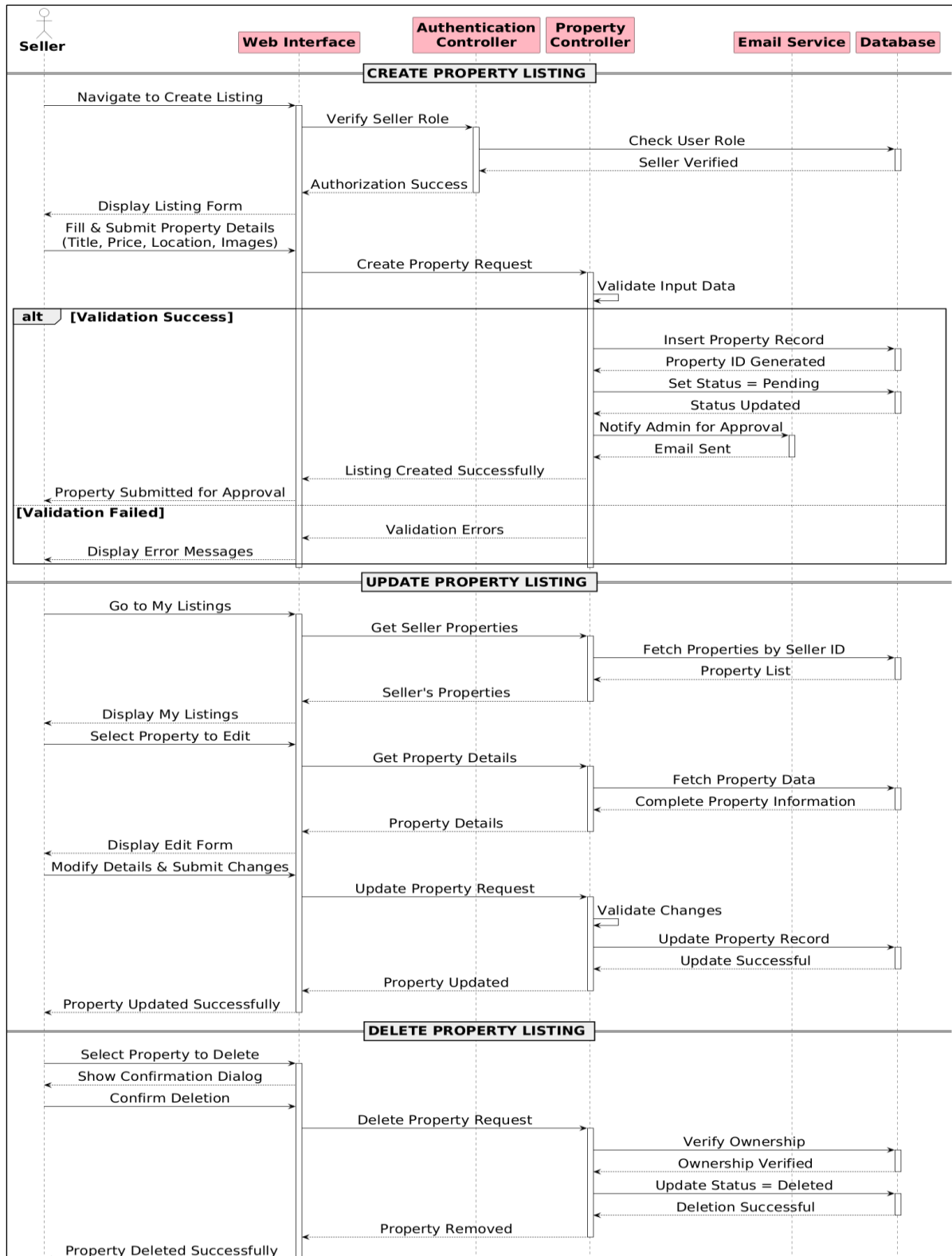


Figure 4.6: Seller Create and Listing of Properties Sequence Diagram

## Admin Sequence Diagram

Figure 4.7 illustrates the complete Admin-side interaction flow within a real estate web application, showing how the admin communicates with various system components including the authentication system, property management module, user management module, blog management module, database, and notification services.

At the start of the process, the admin enters the system through the web interface by accessing the login page. The credentials provided by the admin are forwarded to the authentication controller. The system then validates the credentials by checking them against the stored records in the database. Along with credential verification, the system also checks the role of the user to ensure that the login attempt is coming from an authorized admin. If the credentials and role are valid, the authentication service grants access and the system loads the admin dashboard. If not, access is denied and an error message is returned. Once the admin successfully reaches the dashboard, they can manage multiple modules of the system. In the property management workflow, the admin selects the option to view all pending property listings. This request is sent from the web interface to the property controller, which queries the database to retrieve all properties with a “pending” status. The retrieved list is then displayed on the admin interface. The admin can review each property’s details such as location, price, images, and description. After evaluation, the admin can either approve or reject a property. If the property is approved, the system updates its status in the database from “pending” to “active.” This change ensures that the property becomes visible to all users on the public platform. After successful approval, the notification or email service is triggered to inform the property owner that their listing has been approved and is now live.

In the user management module, the admin can view all registered users by sending a request through the interface. The user controller fetches all user records from the database and returns them to the admin panel. The admin can inspect user profiles, monitor activity, and manage account roles such as buyer, seller, or admin. If the admin updates any user information such as role changes, account activation, or deactivation, the updated data is sent back to the controller. The system then updates the corresponding record in the database and confirms the update operation to the admin interface. In the blog management module, the admin has full control over blog content displayed on the website. The admin can retrieve all existing blog posts through a request to the blog controller, which fetches data from the database. For content creation, the admin enters new blog details, which are then inserted into the database and published on the platform. For editing, the selected blog post is updated with new content, and changes are saved in the database. For deletion, the selected post is removed permanently, and the system confirms successful deletion to the admin. Throughout all these operations, the database plays a central role by storing and retrieving all system data, while controllers act as intermediaries between the user interface and backend logic. The email or notification service

ensures communication between the system and users whenever important actions such as approvals or updates occur.

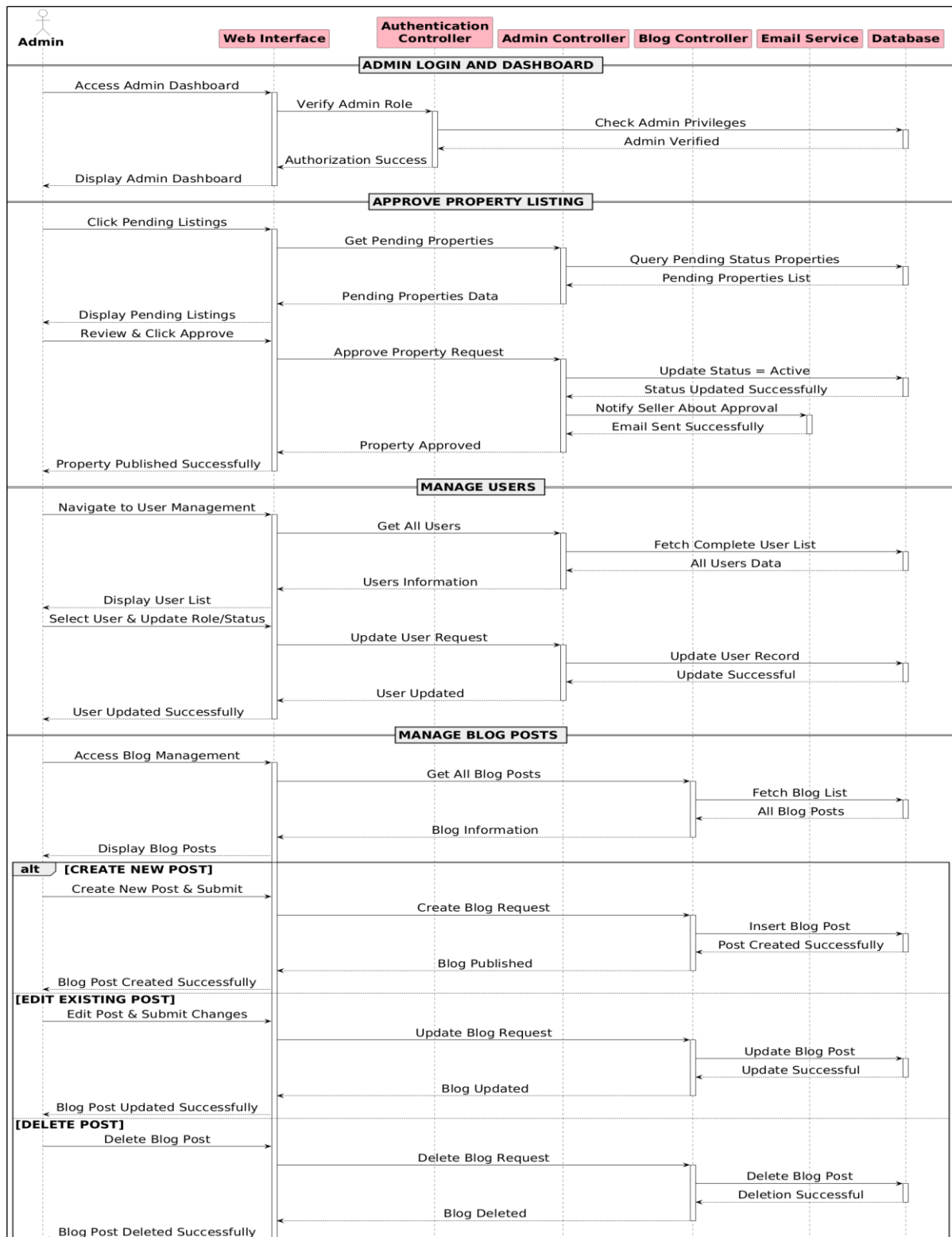


Figure 4.7: Admin Operations Sequence Diagram



### AI Chatbot Query Processing Sequence Diagram

Figure 4.8 models the interaction flow for a multi-purpose chatbot, detailing how user queries are processed for property searches, blog content, price predictions, and general information. This sequence diagram represents the working flow of a **real estate chatbot system**, showing how a user interacts with the system and how different components collaborate to process various types of queries.

The interaction begins when the user opens the chatbot through the web interface. The system displays a chat window where the user enters a query or question. This query is then sent to the chatbot service, which acts as the central processing unit of the system. The chatbot analyzes the user's input using Natural Language Processing (NLP) to understand the intent behind the query.

Based on the identified intent, the system follows different alternative flows. If the query is related to property search, the chatbot forwards the request to the property controller. The property controller interacts with the database to retrieve relevant property data. Once the results are fetched, they are sent back to the chatbot, which then presents suitable property recommendations to the user.

If the user asks for general information, the chatbot communicates with the blog controller. The blog controller queries the database for relevant blog posts or articles. These articles are returned to the chatbot, which provides the user with useful information along with links to related content.

In the case of a price prediction query, the chatbot sends a request to the AI price prediction service. This service processes the request using predictive models and returns an estimated price range. The chatbot then delivers this price estimation to the user.

For location or area-related queries, the chatbot directly queries the database to obtain location-specific details. The retrieved information is then shared with the user in a structured format.

If the user asks for help or frequently asked questions, the chatbot retrieves predefined responses and provides immediate assistance without needing to query other services.

Additionally, the system logs each interaction for tracking and improvement purposes. The conversation details are stored in the database after processing each query.

After receiving a response, the user can choose to continue the conversation by asking another question, in which case the same processing cycle repeats. If the user decides to end the chat, the session is closed and the complete chat history is saved in the database.

Overall, the diagram illustrates a modular and scalable architecture where the chatbot intelligently routes user queries to appropriate services and ensures efficient and accurate responses.

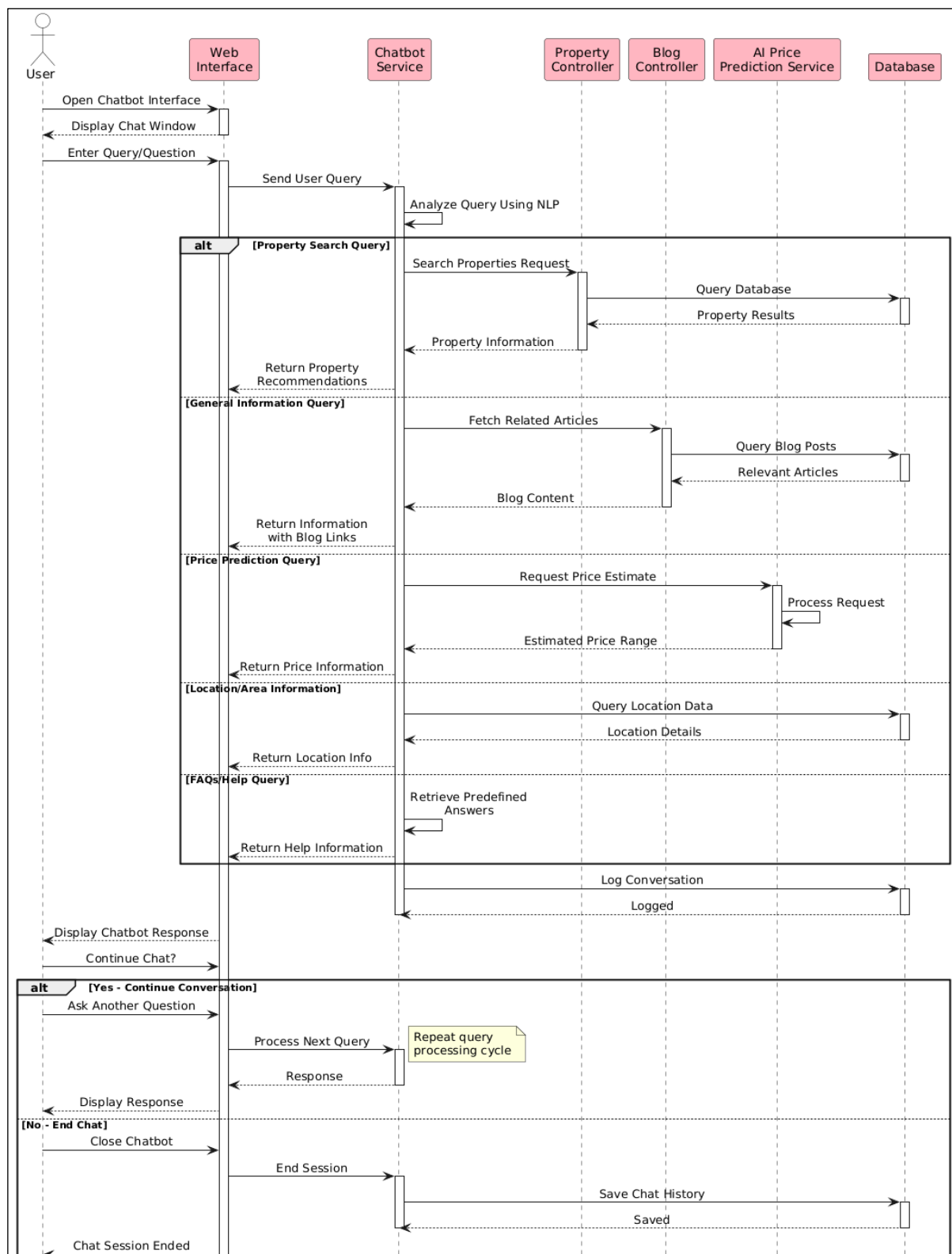


Figure 4.8: AI Chatbot Query Processing Sequence Diagram

#### 4.4.2 Class Diagram

Figure 4.9 represents a class diagram of a real estate system that manages users, properties, interactions, and supporting services like chatbot and AI prediction. It shows how different classes are structured and how they are related to each other.

At the core of the system is an abstract class called **User**, which contains common attributes such as `userId`, `username`, `email`, `password`, `phone`, `profile image`, and `registration date`. It also provides basic functionalities like registration, login, logout, and profile update. This class is inherited by three main roles: Buyer, Seller, and Admin. The Buyer class includes features such as searching properties, contacting sellers, adding properties to favorites, and submitting comments. The Seller class is responsible for managing property listings, including adding, editing, deleting properties, and responding to buyer inquiries. The admin class has higher-level control and is responsible for managing users, approving properties, moderating comments, and managing blog content.

The Chatbot class represents an automated interaction system that allows buyers to communicate with the platform. A buyer can initiate multiple chatbot sessions, and the chatbot handles sending messages, generating responses, and ending conversations.

The Property class is a central entity in the system, containing details such as title, description, price, area, number of bedrooms and bathrooms, type, and status. Each property is listed by a seller and is associated with exactly one location. The Location class stores geographical details like city, state, and coordinates. A property can have multiple images managed by the PropertyImage class, and optionally a 3D view represented by the Property3DView class.

The system also supports user interactions. The Favorite class allows buyers to save properties for later reference, establishing a many-to-many relationship between buyers and properties. The Contact class facilitates communication between buyers and sellers regarding a specific property, storing messages and timestamps.

For content management, the system includes Blog and News classes. These are managed by the Admin, who can create and publish content. Users can also add comments through the Comment class, which are linked to properties and moderated by the admin before approval.

Additionally, the system incorporates an AIPrediction class that provides intelligent insights such as predicted property prices and market trends. Each property can optionally have one associated prediction generated by the AI service.

Overall, the diagram illustrates a well-structured and modular system where responsibilities are clearly divided among classes, and relationships such as inheritance, association, and aggregation are used to model real-world interactions in a real estate platform. Furthermore, the class diagram highlights the use of object-oriented principles such as encapsulation and abstraction, which improve code reusability and system maintainability. Each class is designed to handle specific responsibilities, ensuring a clear separation of concerns within the system architecture. The relationships between classes, such as one-to-many and many-to-many associations, accurately represent real-world interactions between users and properties. This structured design also supports future scalability, allowing new features to be integrated

[illegible]

65

# **Chapter 5**

## **IMPLEMENTATION**

## 5. Implementation

This chapter explains how the Apni Jgah system is implemented, including the use of algorithms, APIs, database integration, and user interface design. The implementation follows a modular approach, where each feature of the system is developed, tested, and integrated step by step. This ensures that the system remains organized, easy to manage, and scalable for future improvements. Each module, such as user management, property handling, search functionality, Wishlist, and price estimation, is developed independently and then connected to form a complete working system. The development is guided by UML diagrams such as class diagrams and sequence diagrams, which help in visualizing system structure and interactions between different components. Class diagrams define the structure of the system by showing classes, attributes, and relationships, while sequence diagrams illustrate how different parts of the system communicate during specific operations such as user login or property search. These diagrams ensure clarity in design and help in reducing development errors.

The backend of the system is implemented using Django, which handles the core logic, data processing, and communication with the database. Django models are used to define the structure of data such as users, properties, and Wishlist items. Views are responsible for handling user requests and returning appropriate responses, while templates are used to display data on the frontend. The system follows the Model-View-Template (MVT) architecture, which helps in separating concerns and improving maintainability. The frontend is designed using HTML, CSS, and Bootstrap to create a responsive and user-friendly interface. The user interface is kept simple and clean so that users can easily navigate through the system. Forms are used for user input such as registration, login, and property submission. Proper validation is applied to ensure that the data entered by users is correct and complete. The interface is designed to work on different screen sizes, making it accessible on both desktop and mobile devices. APIs and internal routing mechanisms are used to connect different parts of the system. Django URLs are configured to map user requests to the appropriate views. These APIs handle operations such as fetching property data, submitting forms, and managing user sessions. This ensures smooth communication between the frontend and backend components of the system.

Algorithms are applied in different parts of the system to improve functionality and performance. Search and filtering algorithms are used to retrieve properties based on user preferences such as location, price range, and property type. A simple price estimation logic is implemented to provide users with an approximate idea of property value based on selected features. These algorithms help in making the system more efficient and user-friendly. The database is implemented using SQLite or MySQL, where all system data is stored in an organized manner. Tables are created for users, properties, and other related entities. Relationships between tables are properly defined to maintain data integrity. The database ensures that all information is stored securely and can be retrieved efficiently when needed.

. Overall, the implementation of the Apni Jgah system focuses on simplicity, modularity, and efficiency. By using structured design, appropriate technologies, and clear development practices, the system is able to provide a reliable and user-friendly real estate platform

## **5.1 Algorithm**

This section describes the main functions of the Apni Jaga real estate platform. For each function, the purpose, input parameters, output, and algorithm steps are presented. The algorithms are written in simplified pseudocode style to show how each function processes data, validates inputs, and performs its tasks in the system.

### **5.1.1 User Registration**

#### **START UserRegistration**

1. Display form fields.
2. When user clicks “Register”:
  - a) Check if all fields are filled.
- a. IF any field is empty → Show: “Please fill all fields” → STOP
- b) Validate password length (must be 8–12 characters).
- b. IF invalid → Show error → STOP
- c) Check if Password = Confirm Password.
- c. IF not → Show: “Passwords do not match” → STOP
3. Check if email already exists in the database.
- a. IF email exists → Show: “Email already registered” → STOP
4. Hash the password securely.
5. Create a new user record in the database.
6. If registration successful → Redirect user to Login Screen.
7. If any error occurs → Show error message.

#### **END UserRegistration**

### **5.1.2 Authenticate User**

#### **START AuthenticateUser**

1. User enters email and password on login screen.
2. Validate that both fields are filled.
3. Fetch user by email from the database.
  - If user not found → Show “Invalid credentials” → STOP.
4. Compare entered password with hashed password.
  - If incorrect → Show “Wrong password” → STOP.
5. Check if account is active.

- If not active → Show “Account disabled” → STOP.
- 6. Update user's last\_login timestamp.
- 7. Generate a session token for the user.
- 8. Redirect user to Dashboard.

**END AuthenticateUser**

### **5.1.3 Create Property Listing**

**START CreateProperty**

1. Show form to Seller/Agent with fields: Title, Price, Description, Area, Bedrooms, Bathrooms, City/Area/Address, Property Type, Images, Amenities.
2. On Submit:
  - a) Validate required fields
  - b) Check if user has Seller/Agent role → STOP if not
3. Create property with status = Pending Approval
4. Save address and convert to latitude/longitude via Maps API
5. Upload images to cloud storage and save URLs
6. Save selected amenities linked to property
7. Notify admin about new listing
8. Show success message

**END CreateProperty**

### **5.1.4 Search Properties**

1. **START SearchProperties**
2. User selects filters like:
  - City
  - Area
  - Price range
  - Property type
  - Bedrooms
  - Area size
3. System loads only **active** properties from database.
4. Apply filters one by one:
  - a) Location filter
  - b) Price range filter
  - c) Property type filter
  - d) Bedroom/area size filter
5. Sort results by newest first.
6. Paginate results (10–20 per page).
7. Display final results to user.



## **8. END SearchProperties**

### **5.1.5 Get Property Details**

1. **START GetPropertyDetails**
2. User selects a property.
3. Fetch property by ID.
4. Increase view\_count by 1.
5. Fetch related data:
  - Images
  - Address
  - Seller details
6. Display full property page.
7. **END GetPropertyDetails**

### **5.1.6 Add to Favourites**

1. **START AddToFavourites**
2. Check if user is logged in.
  - If not → Show login screen.
3. Get property by ID.
4. Check if already in favourites.
  - If yes → Show message “Already added”.
5. Create new Favourite record.
6. Show success message.
7. **END AddToFavourites**

### **5.1.7 AI Price Prediction**

1. **START GeneratePricePrediction**
2. Extract property features (location, size, bedrooms, bathrooms, etc.).
3. Normalize/scale data.
4. Load pre-trained ML model.
5. Run prediction to get price.
6. Calculate confidence score.
7. Save prediction result in database.
8. Show predicted price to the user.
9. **END GeneratePricePrediction**

### **5.1.8 Process Chatbot Query**

1. **START ProcessChatQuery**
2. User sends a chat message.
3. Get or create conversation session.
4. Save user message.

5. Analyze text using NLP to detect intent.
6. Generate chatbot response.
7. Save chatbot reply.
8. Return answer to user.
9. **END ProcessChatQuery**

#### **5.1.9 Update Property**

1. **START UpdateProperty**
2. Fetch property by ID.
3. Verify logged-in user is the owner.
4. Validate updated data.
5. If address changed → geocode new location.
6. Save updated details.
7. Show success message.
8. **END UpdateProperty**

#### **5.1.10 Delete Property**

1. **START DeleteProperty**
2. Fetch property by ID.
3. Verify ownership.
4. Update status to **Deleted** (not removed physically).
5. Show confirmation.
6. **END DeleteProperty**

#### **5.1.11 Create Blog Post (Admin)**

7. **START CreateBlogPost**
8. Verify admin access.
9. Validate title, content, category.
10. Create blog post and set publish date.
11. Save post.
12. Show success message.
13. **END CreateBlogPost**

#### **5.1.12 Send Notification**

1. **START SendNotification**
2. Create notification record with:
  - User
  - Message
  - Type (email/push/system)
3. Save notification.
4. If email → Send email to user.

5. Return notification result.
6. **END SendNotification**

## 5.2 External APIs

External APIs (Application Programming Interfaces) are pre-built services provided by third-party platforms that allow our system to access powerful features without developing them from scratch. These APIs help in extending system functionality, improving performance, and providing real-time data integration. In a real estate platform, external APIs play an important role in enhancing user experience by supporting map services, location tracking, currency conversion, and future intelligent predictions. They also make the system more flexible, scalable, and easier to maintain. By using external APIs, the system becomes more interactive and user-friendly because users can easily view property locations on maps, find nearby listings, and understand prices in their preferred currency. Additionally, these APIs ensure that the system remains compatible with global users and can be upgraded in the future with advanced machine learning capabilities.

The following table shows the external APIs used in the system and their purposes:

*Table 5.1: External APIs*

| Name of API       | Description of API                                            | Purpose of Usage                                                             | Function/Class Name in which Used                |
|-------------------|---------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------|
| Google Maps API   | Provides location and interactive map services                | Display property locations, show maps, and help users find nearby properties | map.js (JavaScript for frontend map integration) |
| Geolocation API   | Detects user's current location using browser/device services | Identify user location to show nearby properties and personalized results    | navigator.geolocation (JavaScript in frontend)   |
| OpenStreetMap API | Open-source mapping service used as an alternative            | Used as backup map service if Google Maps is unavailable                     | map_fallback.js (JavaScript frontend file)       |
| Future ML API     | Provides machine learning-based prediction models             | Predict property prices and improve decision-making accuracy                 | prediction_view (Django view / backend logic)    |

In addition to the APIs listed above, the integration of external services significantly improves the overall efficiency and intelligence of the *Apni Jgah* platform. These APIs reduce development effort by providing ready-made solutions for complex functionalities such as mapping, geolocation, and real-time data processing.

From a system design perspective, external APIs also support modularity and scalability. Each API is integrated as a separate service module, which means that if one API needs to be updated or replaced, it does not affect the entire system. For example, if the Google Maps API service becomes unavailable or costly, the system can automatically switch to the OpenStreetMap API without disrupting user experience.

Moreover, the use of APIs enhances real-time data accuracy. The Currency API ensures that property prices are always up-to-date according to global exchange rates, which is particularly beneficial for overseas users interested in investing in local real estate. Similarly, the Geolocation API allows the system to dynamically adjust search results based on the user's current location, making property searches more relevant and personalized.

Security and reliability are also considered when integrating external APIs. All API requests are handled through secure protocols (HTTPS), and proper error handling mechanisms are implemented to manage failures, timeouts, or invalid responses. This ensures that the system remains stable even when external services experience issues.

In future enhancements, more advanced APIs can be integrated, such as AI-based recommendation systems, which suggest properties based on user behavior, preferences, and search history. Additionally, analytics APIs can be used to track user interactions and improve system performance over time.

Overall, external APIs play a crucial role in transforming the *Apni Jgah* platform into a modern, intelligent, and user-friendly real estate system by enabling advanced features, improving system flexibility, and supporting future technological growth.

### **5.3 User Interface**

The user interface (UI) of the Real Estate System is designed to provide a simple, modern, and user-friendly experience for all types of users including buyers, sellers, agents, and administrators. The main focus of the UI is on ease of use, clarity, and fast navigation so that users can easily search properties, view detailed listings, compare options, and manage their accounts without any difficulty. The interface follows a clean layout structure with well-organized menus, clear labels, and intuitive icons that guide users throughout the system. The design follows modern web UI principles with responsive layouts, ensuring that the system works smoothly on desktops, tablets, and mobile devices. Important features such as property search, filters, maps integration, and user dashboards are placed in easily accessible sections to improve usability. Consistent color themes, readable typography, and proper spacing are used to enhance visual clarity and user engagement. Additionally, interactive elements such as pop-up messages, confirmation dialogs, and error alerts provide real-time feedback to improve user control and reduce mistakes. Overall, the UI is

developed to improve user experience, reduce complexity, and ensure fast and efficient interaction with the system.

## Homepage

The Home Page shown in Figure 5.1 of the User Application acts as the main entry point of the system. It provides quick access to all major features such as browsing properties, searching listings, viewing property details, and accessing user profiles.

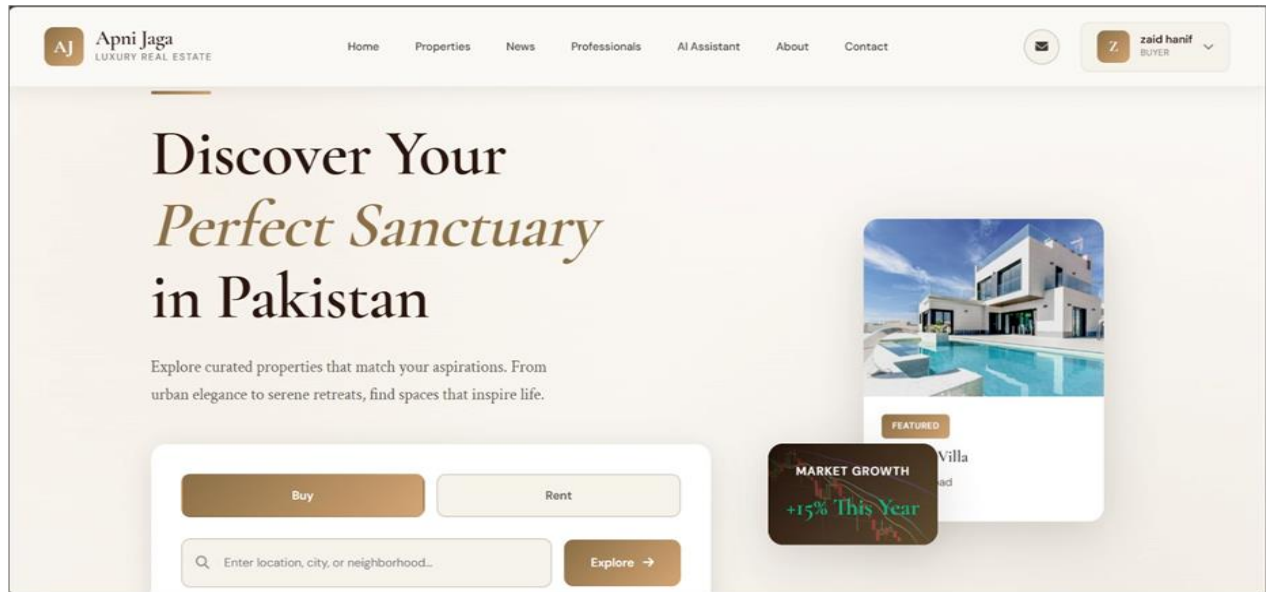
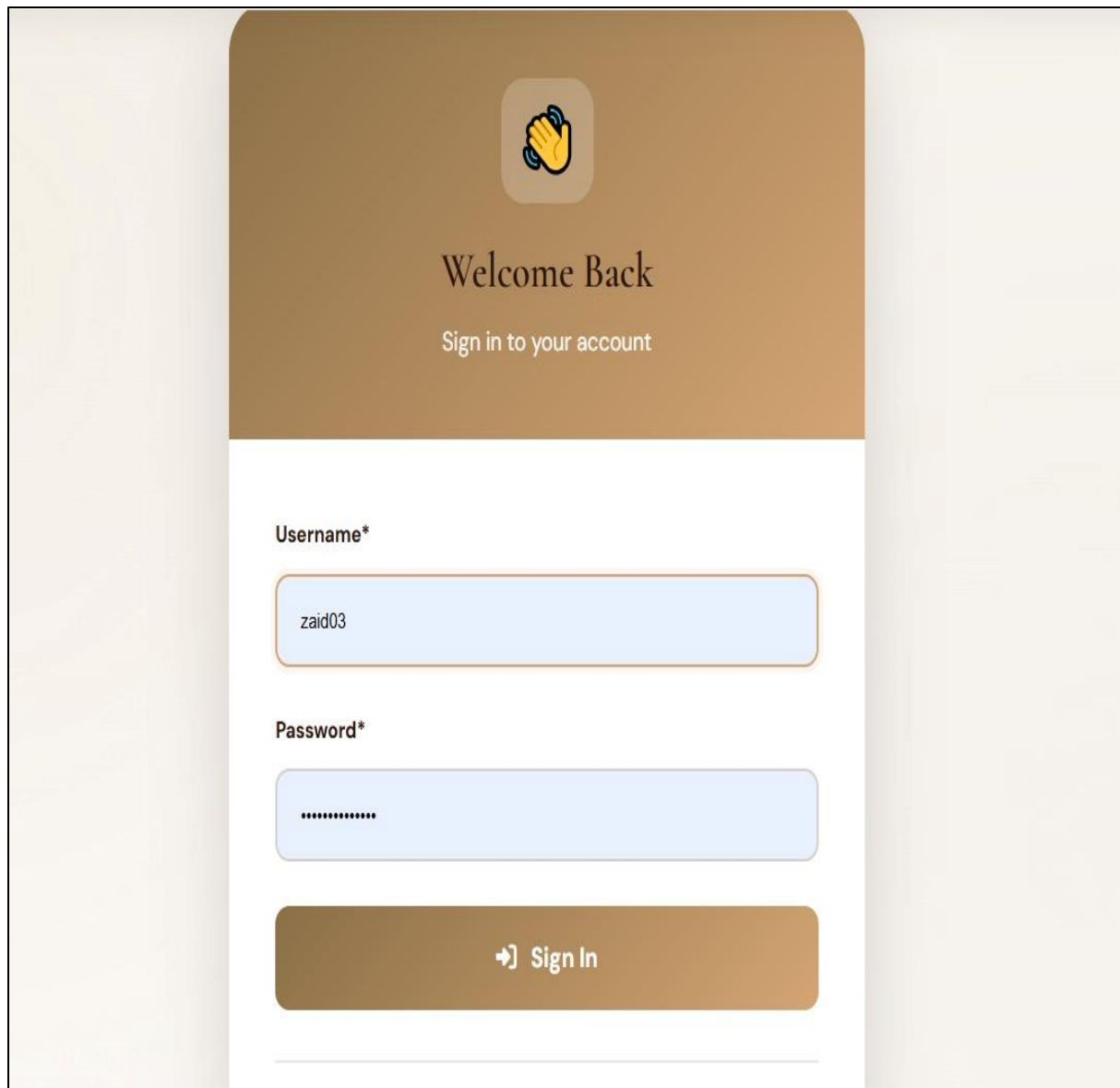


Figure 5.1:Homepage

## Login Page

Figure 5.9 shows a simple and user-friendly login interface where users can easily enter their username and password to access their accounts. The layout is clean and well-organized, making it easy to use for all types of users. A “Sign up” link is also provided for new users to create an account.

The system includes basic validation and security features to ensure only authorized users can log in. After successful login, users can access personalized features such as their dashboard and saved (Wishlist) properties, making the experience more convenient and efficient.

The image shows a login page with a brown header section. At the top center of the header is a yellow hand icon with blue motion lines. Below the icon, the text "Welcome Back" is displayed in a large, dark font, followed by "Sign in to your account" in a smaller, lighter font. The main content area is white and contains two input fields. The first is labeled "Username\*" and contains the text "zaid03". The second is labeled "Password\*" and contains a series of dots. Below these fields is a large brown button with a right-pointing arrow icon and the text "Sign In".

Welcome Back

Sign in to your account

Username\*

zaid03

Password\*

.....

➔ Sign In

*Figure 5.2: Login Page*

## **Property Browsing Screen**

The Property Browsing Page, as shown in Figure 5.3 and Figure 5.4, provides a well-structured and user-friendly interface that allows users to easily explore available properties. In the main section, a prominent heading encourages users to find their ideal property, along with a brief description highlighting available listings. Below this, a search panel is provided where users can enter details like location, property type, and price range to refine their search, enabling users to quickly find properties by entering keywords such as location or property name. In addition, filter options are provided to refine results based on criteria like property type, making the search more specific and efficient.

Property listings are displayed in neatly designed cards, where each card presents key details such as price, location, and basic property information. Images are also included to give users a quick visual overview of the property. This card-based layout makes the interface visually appealing and easy to understand.

The design supports quick comparison between multiple properties, as users can view important details at a glance without opening each listing individually. Users can also click on a specific property card to view more detailed information. Overall, the page is designed to ensure smooth navigation, better organization of data, and a convenient browsing experience for users.

The screenshot shows the top section of the Apni Jaga Real Estate website. The header includes the logo 'Apni Jaga LUXURY REAL ESTATE' on the left, a navigation menu with links 'Home', 'Properties', 'News', 'Professionals', 'AI Assistant', 'About', and 'Contact' in the center, and a user profile 'Sawera Tubessam BUYER' on the right. Below the header is a large banner with the text 'Discover Your Perfect Property' and a subtitle 'Explore our curated collection of premium properties across Pakistan'. At the bottom of the banner is a 'Refine Your Search' section with four filters: 'LOCATION' (text input 'City, area or neighborhood'), 'TYPE' (dropdown 'All Types'), 'PROPERTY TYPE' (dropdown 'All Properties'), and 'MIN PRICE' (text input 'Minimum'). A 'Filters' button is located to the right of the search filters.

*Figure 5.3: Search Property*

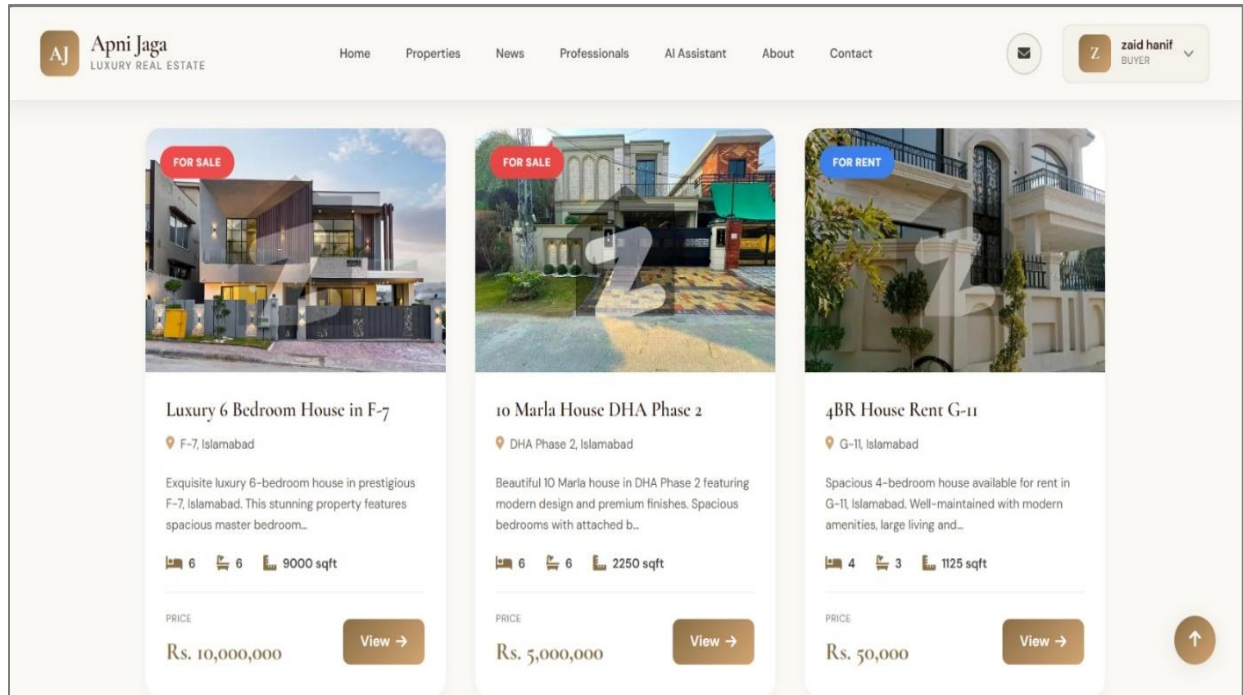


Figure 5.4: Property Browsing Page

## Chatbot

Figure 5.5 and Figure 5.6 shows chatbot interface provides AI-powered property assistance, allowing users to ask questions via buttons or custom queries. It displays relevant listings with key details in a clean, organized, and engaging conversational format.

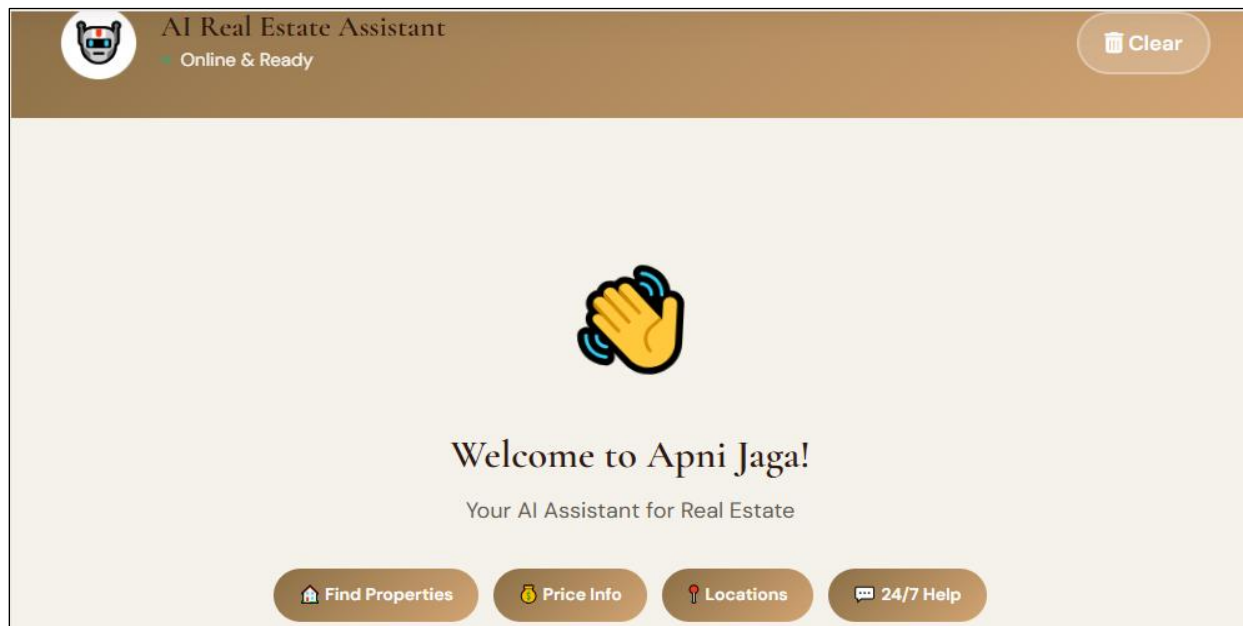


Figure 5.5: Chatbot





Figure 5.6: Chatbot Query

## Professional Workers Screen

Figure 5.7 shows professional workers professionals section allows users to connect with verified property professionals. A clear header and sidebar filters help users browse categories like engineers and designers, making it easy to find specialized expertise for real estate projects.

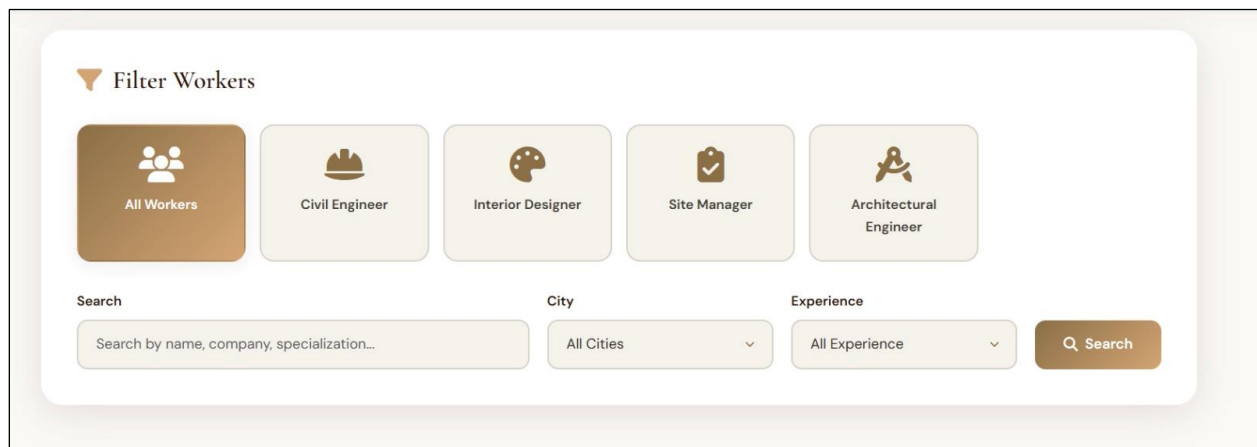


Figure 5.7: Professional Workers

## Professionals Profiles

Figure 5.8 displays verified and featured real estate professionals, highlighting their expertise, experience level, skills, ratings, and location. Each profile includes a short bio describing their specialization and achievements. Users can view detailed profiles or directly contact the professionals for their services. Hourly rates, reviews, and profile badges help users make informed decisions.

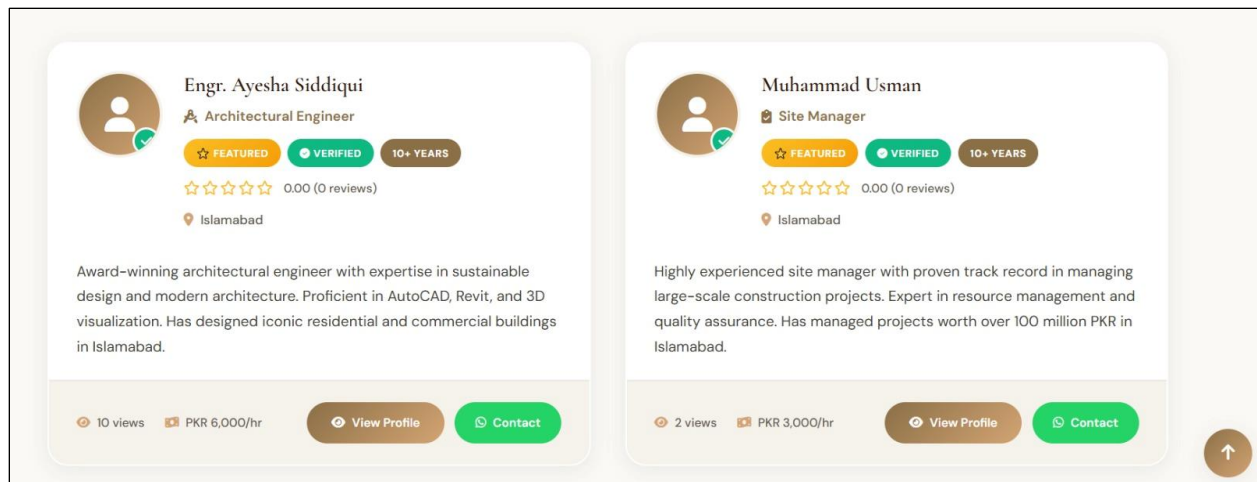


Figure 5.8: Professionals Profiles

## My Property Page

Figure 5.9 shows My Properties Dashboard lets property owners see and manage all their listings in one place. They can easily add, edit, or delete properties, making management simple and convenient.

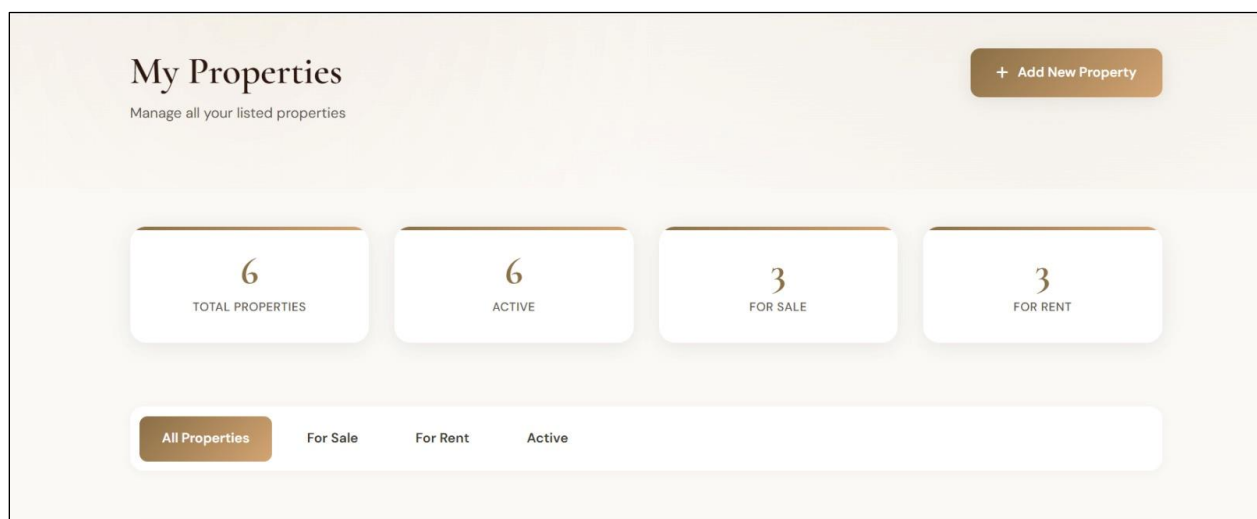


Figure 5.9: My Property Page

## 3D Virtual Tour Screen

Figure 5.8 shows a popup for a **3D Virtual Tour** feature on a real estate website. It offers three subscription plans (1, 3, and 6 months) with different prices and discounts. Users are instructed to complete the booking by sending a payment screenshot via WhatsApp.

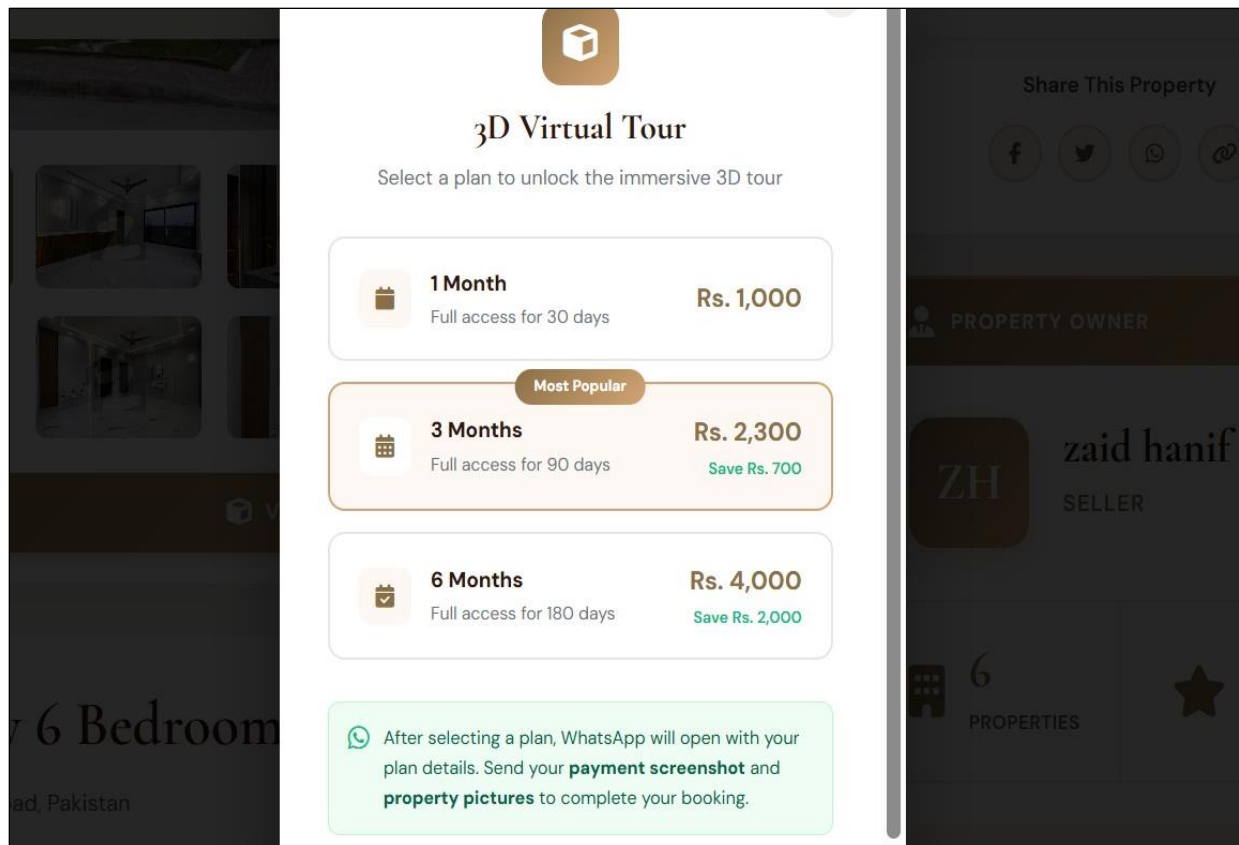


Figure 5.10: 3D Virtual Tour

## Contact Us Screen

Figure 5.11 represents Contact Us screen in this project allows users to easily communicate with the system administration regarding any queries, feedback, or issues related to the platform. It provides a simple and user-friendly form where users can enter their name, email address, subject, and message.

This screen ensures smooth interaction between users and the system by enabling direct communication without any complexity. Proper validation is applied to ensure that all required fields are filled correctly before submission, which helps maintain the accuracy of user queries.

After submission, the message is sent to the admin panel for review and response. This feature improves user support and enhances the overall usability of the system by providing an efficient way to handle user concerns and feedback

Apni Jaga LUXURY REAL ESTATE

Home Properties News Professionals AI Assistant About Contact

Sawera Tubessam BUYER

### Send Us a Message

Your Name \* Email Address \*

Phone Number Subject \*

Message \*

### Contact Information

Office Address  
123 Main Street, Gulberg III  
Lahore, Punjab 54000  
Pakistan

Phone Number  
+92 325 1061130  
+92 302 3910072

Figure 5.11: Contact Screen

## News Screen

Figure 5.12 shows the News section of the system, where the latest updates, announcements, and relevant information are displayed for users. This section is designed to keep users informed about new activities, system updates, and important notices related to the platform. The news content is managed by the admin, allowing them to add, update, or remove news entries as needed. Each news item typically includes a title, short description, and publication date for better clarity and organization. This section improves user engagement by providing timely information and ensuring that users stay updated with the latest developments within the system.

Apni Jaga LUXURY REAL ESTATE

Home Properties News Professionals AI Assistant About Contact

Sawera Tubessam BUYER

## Real Estate News Hub

Stay Updated with Latest Market Trends & Property Insights

Search articles, topics, keywords... Search

All Categories

Figure 5.12: News Screen

## About Screen

Figure 5.13 represents about us module of the Apni Jaga Luxury Real Estate website. It is designed to introduce the platform's core purpose and values to its visitors. The module presents the fundamental goals and vision of the Apni Jaga platform, explaining what the system stands for and what it aims to achieve in the real estate industry of Pakistan. It highlights the platform's commitment to providing a transparent, efficient, and technology-driven environment for property dealings.

The module also emphasizes the role of artificial intelligence and modern technology in simplifying the property searching experience for users. Visually, the module is supported by a relevant real estate image that reinforces the theme of home ownership and property dealing. Overall, this module serves as an informational and motivational section of the website that builds trust and credibility among visitors by clearly communicating the mission and values of the Apni Jaga platform, helping users understand the purpose behind the system before they begin exploring properties or using any of its features.

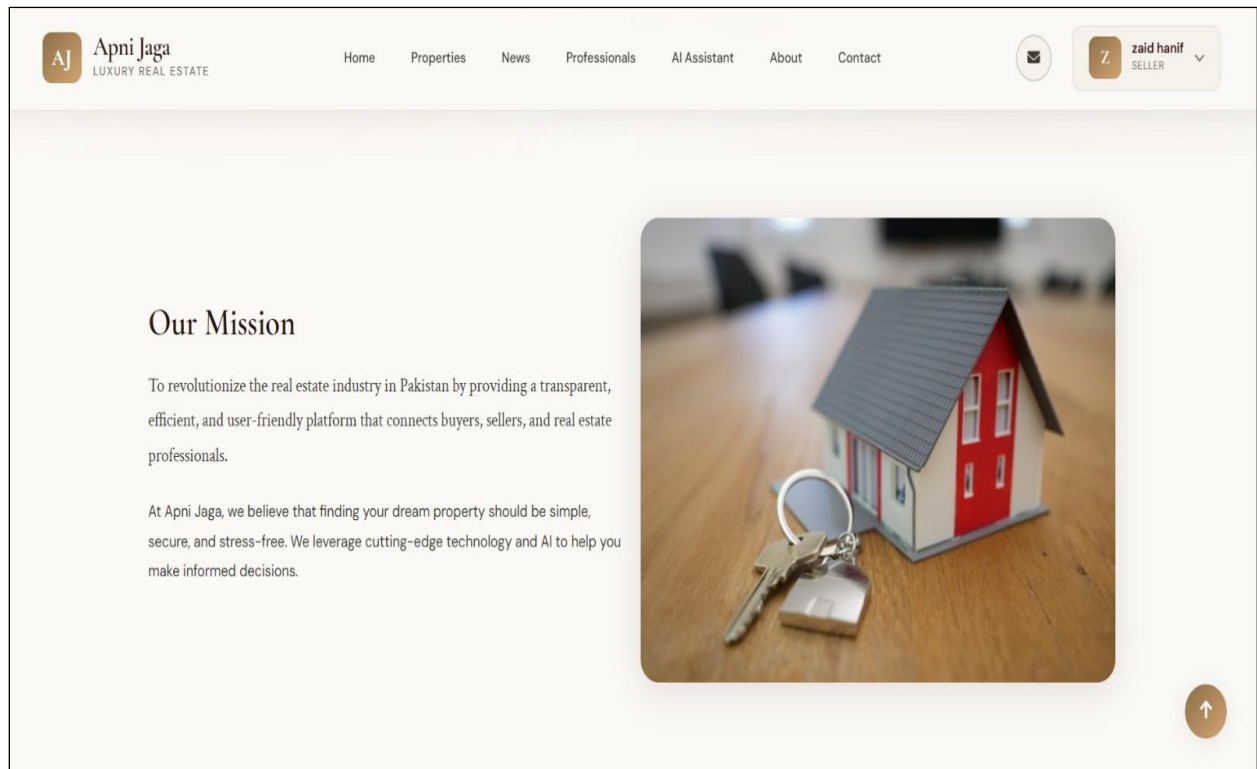


Figure 5.13: About Us

## Favorites Screen

Figure 5.14 shows the Favorites Screen of the *Apni Jagah* real estate platform, which allows users to view and manage the properties they have marked as favorite. This screen provides a convenient way for users to keep track of selected properties without searching for them again.

In this screen, all saved properties are displayed in a well-organized card layout. Each property card contains essential information such as property image, title, location, price, and key details like area size and number of bedrooms. This structured presentation helps users quickly review multiple properties at a glance.

The screen also provides interactive options for users. They can view detailed information of any property, remove a property from the favorites list, or directly contact the seller or agent. A heart icon or remove button is included on each card to manage favorites easily.

Additionally, the Favorites Screen is linked to the user account, ensuring that saved properties remain available even after logging out and logging back in. If any property becomes unavailable, it is either removed or clearly indicated to maintain data accuracy.

Overall, this screen enhances user experience by providing quick access to preferred listings and supporting efficient decision-making.

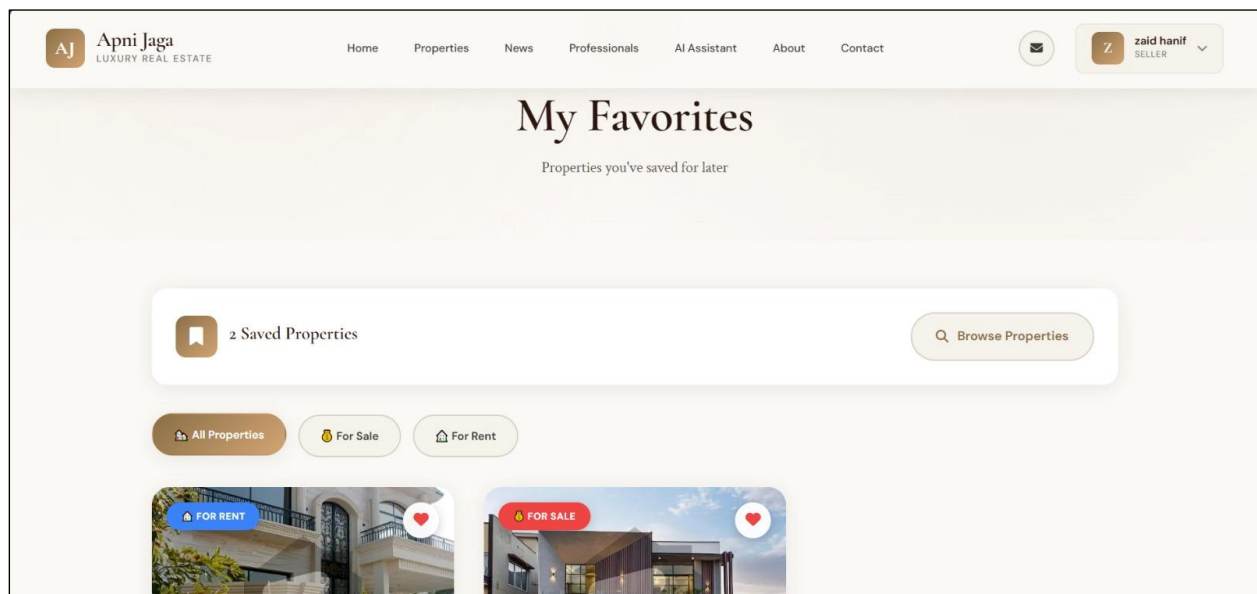


Figure 5.14: Favorites

## Buyer Profile

Figure 5.15 displays the buyer's personal identity, including their name and username, along with a profile picture placeholder. A key feature of this page is the **verification status**, which confirms that the buyer's account has been authenticated by the system, increasing credibility and trust for interactions with sellers and agents. The **buyer role badge** clearly identifies the user's role, ensuring that the system provides relevant features and access tailored to property purchasing activities.

This page also provides account management functionalities. The "Edit Profile" option allows the buyer to update personal details such as name, contact information, or profile image, while the "Change Password" feature ensures account security and privacy. These controls give users full ownership of their account information.

From a functional perspective, this profile acts as a central point for the buyer's interaction with the system. Through the navigation bar, the buyer can easily access property listings, view real estate news, connect with professionals, or use AI-based assistance for property-related queries. The design focuses on simplicity and usability, enabling buyers to efficiently manage their profile while seamlessly navigating the platform to search and evaluate properties.

Overall, this buyer profile page plays a vital role in maintaining user identity, ensuring secure access, and supporting the buyer's journey from property search to potential purchase within the system.

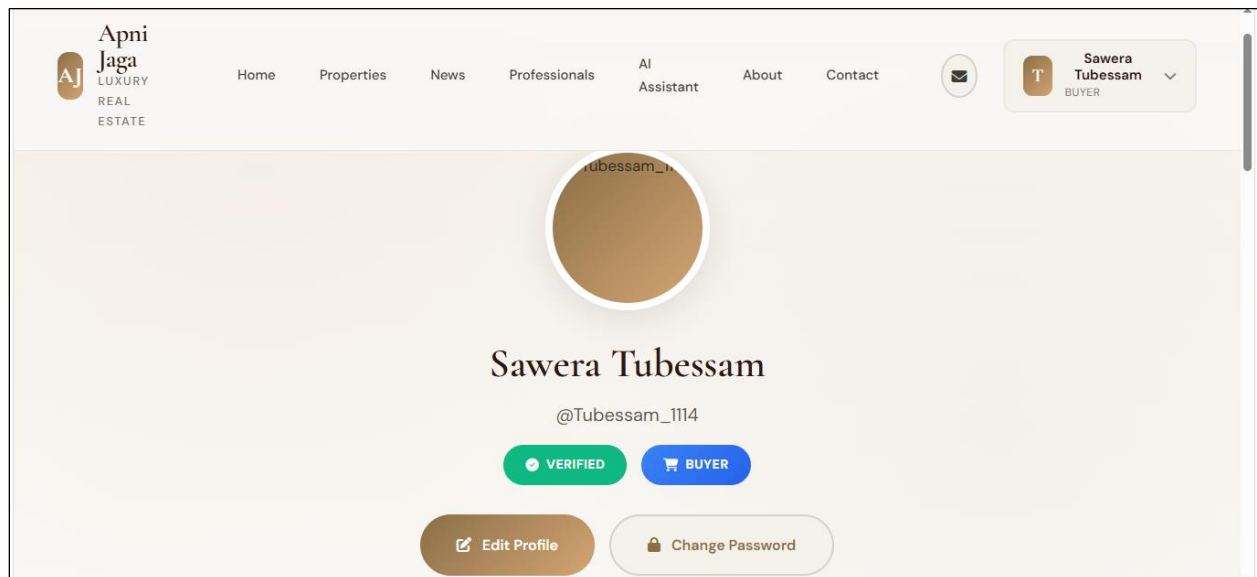


Figure 5.15: Buyer Profile

## Seller Profile

Figure 5.16 represents the seller profile module of the Apni Jaga real estate system. It is specifically designed for users who are registered as sellers and are responsible for listing and managing properties on the platform.

The profile prominently displays the seller's personal information, including their name and username, along with a profile picture placeholder. A verification badge is shown to indicate that the seller's account has been authenticated, which enhances credibility and builds trust among potential buyers. The seller role indicator clearly defines the user's role, ensuring that the system provides features relevant to property listing and selling activities.

This page also includes essential account management options. The "Edit Profile" button allows the seller to update personal details such as contact information and profile image, while the "Change Password" option ensures account security. These features give the seller control over their account and help maintain privacy. Functionally, this profile acts as a central hub for the seller's activities within the system. Through the navigation bar, the seller can access different modules such as property listings, where they can add, update, or manage their properties, as well as explore news, connect with professionals, and use AI assistance. The design of the page focuses on clarity and ease of use, enabling sellers to efficiently manage their profile and perform their tasks.

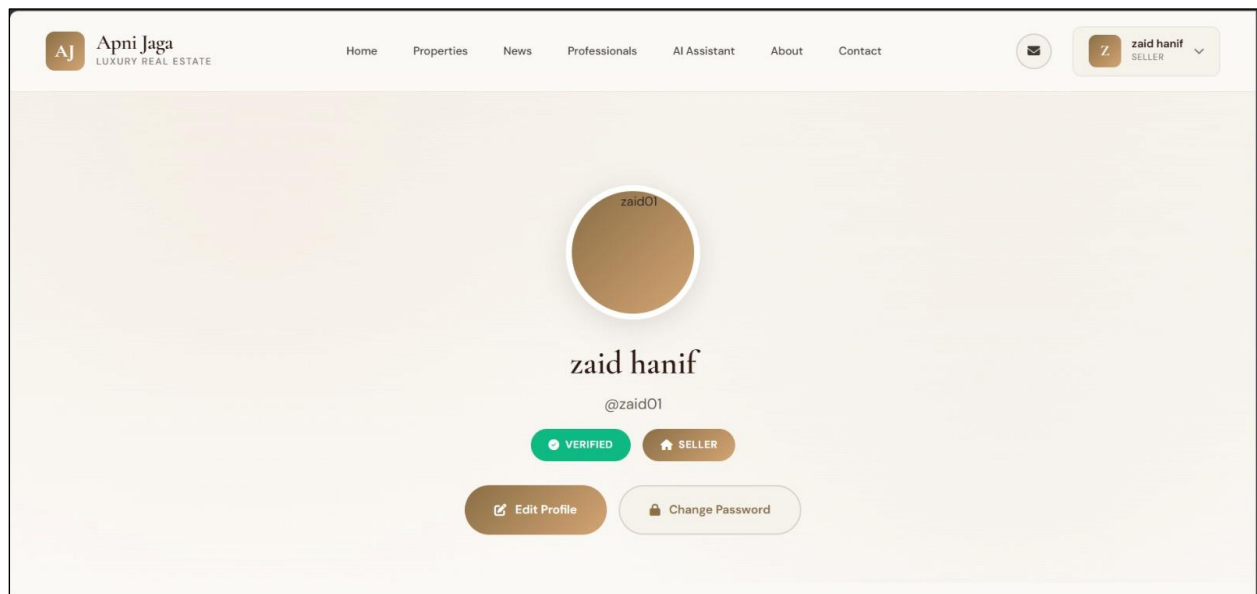


Figure 5.16: Seller Profile



## Professional Profile

Figure 5.17 represents the professional profile module of the Apni Jaga real estate system. It is specifically designed for verified professionals such as architects, engineers, and property experts who offer their services through the platform. The profile prominently displays the professional's identity, including their name, designation (e.g., Architectural Engineer), and a profile picture with a verification badge. It also includes a rating system, showing user feedback and reviews, which helps potential clients evaluate the credibility and quality of the professional's services.

Key highlights such as “Featured Professional”, verification status, and years of experience (e.g., 10+ years) are clearly indicated using visually distinct badges. These elements enhance trust and help users quickly identify experienced and reliable professionals. The page also provides direct communication options, including WhatsApp, Call Now, and Email buttons, enabling users to easily contact the professional for inquiries or service requests. This ensures smooth and immediate interaction between clients and professionals.

Overall, this professional profile page serves as a comprehensive overview of a service provider's qualifications, experience, and credibility, while also facilitating direct communication. It plays a crucial role in connecting buyers or sellers with trusted experts within the platform.

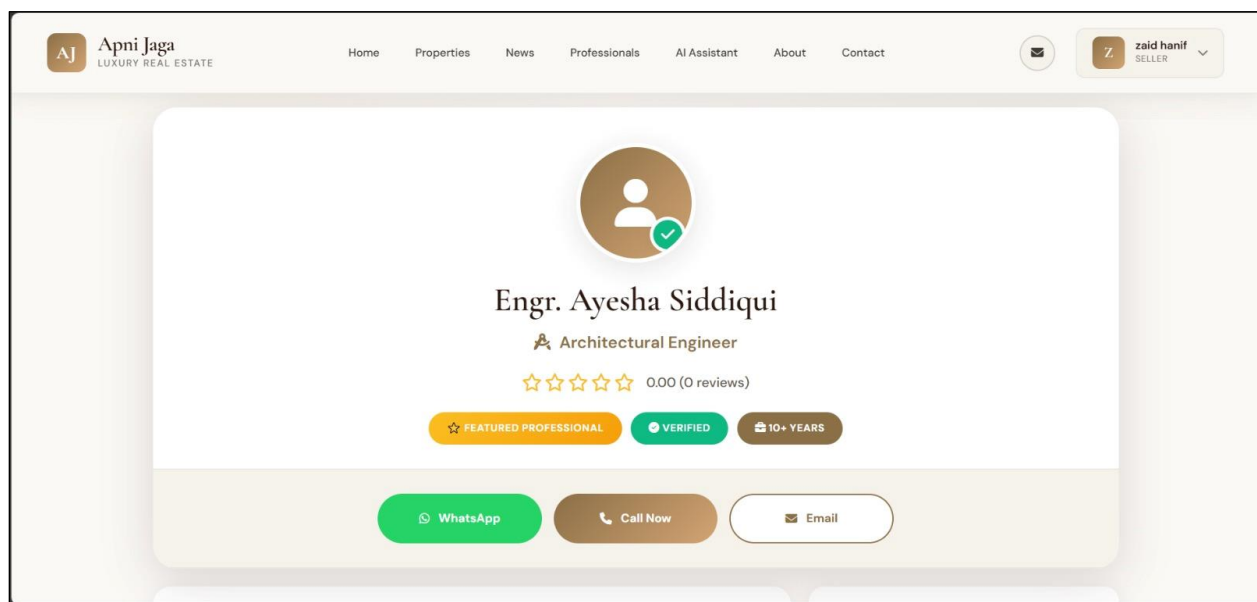


Figure 5.17: Professional Profile

# **Chapter 6**

## **TESTING AND EVALUATION**

## **6. Testing and Evaluation**

This chapter demonstrates that the Real Estate Management System functions correctly, meets the specified requirements, and provides a reliable and secure experience for users. Testing and evaluation are critical phases in the development of the Real Estate Management System, ensuring that the application operates correctly, efficiently, and securely under different conditions. This chapter validates that the system fulfills all functional and non-functional requirements defined earlier, including usability, performance, reliability, and security.

The testing process was carefully planned and executed to detect errors, remove inconsistencies, and verify that all modules perform as expected. Both manual and automated testing approaches were used to achieve comprehensive system validation. Manual testing enabled direct human interaction with the system to identify usability issues, interface problems, and logical errors. On the other hand, automated testing improved efficiency by executing repetitive test cases quickly and consistently without human intervention.

Through this combined testing strategy, the system has been evaluated in terms of correctness, performance, and user satisfaction. The results confirm that the application is stable, responsive, and ready for deployment in a real-world environment.

### **6.1 Manual Testing**

Manual testing is a process in which testers interact directly with the system without using automated tools. It plays a vital role in identifying issues that require human judgment, such as user interface design, navigation flow, and overall user experience. Unlike automated testing, manual testing allows testers to explore the system in a flexible way and detect unexpected behaviors that may not be covered in predefined scripts.

In the Real Estate Management System, manual testing was conducted at multiple levels to ensure thorough validation. These levels include system testing, unit testing, functional testing, and integration testing. Each level focuses on a specific aspect of the system, ranging from individual modules to the complete system as a whole. This layered approach ensures that errors are detected early and resolved before affecting other components.

#### **6.1.1 System Testing**

System testing is a comprehensive evaluation performed after the complete development and integration of all system modules. It verifies that the entire application functions correctly as a unified system and meets all specified requirements. This testing simulates real-world usage scenarios, allowing testers to observe how the system behaves under normal and extreme conditions.

The goal of system testing is not only to check functionality but also to evaluate performance, security, and reliability. It ensures that all modules interact properly and that the system delivers a consistent and smooth user experience.

#### **6.1.1.1 Purpose of System Testing**

The main purpose of system testing is to validate that the system satisfies all functional and non-functional requirements. It ensures that the application performs accurately, handles user requests efficiently, and maintains data integrity. Additionally, it verifies that the system is ready for deployment and can operate reliably in a real-world environment.

#### **6.1.1.2 Types of System Testing**

System testing includes several important types of testing, each focusing on a different aspect of the system:

- Unit Testing – Tests individual modules
- Functional Testing – Tests system features
- Integration Testing – Tests interaction between modules

#### **6.1.1.3 Importance of Manual System Testing**

Manual system testing is essential for ensuring overall system quality. It helps detect errors that automated tests might miss, especially those related to user experience and interface design. It also improves system reliability, enhances security by identifying vulnerabilities, and ensures that the system meets user expectations effectively.

### **6.1.2 Unit Testing**

Unit testing is the process of testing individual components or modules of the system in isolation. Each module is tested separately to ensure that it performs its intended function correctly without depending on other modules. This type of testing is usually performed during the development phase to identify and fix errors early.

In this project, unit testing was conducted on key modules such as login, user profile management, property handling, search functionality, and notification services. Each module was provided with specific input data, and the output was compared with the expected results. This helped ensure that each unit behaves correctly and produces accurate results.

Unit testing is particularly important because it forms the foundation for higher-level testing. If individual modules are not functioning properly, it can lead to failures during integration and system testing. The following table presents the unit testing scenarios performed in this system.

**Table 6.1: Unit Testing**

| Test ID | Module Name         | Test Case/Test Script | Description                   | Input Data                     | Expected Result               | Actual Result        | Status |
|---------|---------------------|-----------------------|-------------------------------|--------------------------------|-------------------------------|----------------------|--------|
| UT01    | Login Module        | Verify login          | Test user login functionality | Username: U001, Password: 1234 | User logs in successfully     | Login successful     | Pass   |
| UT02    | Profile Module      | Edit profile          | Update user profile details   | Email, Phone                   | Profile updated               | Updated successfully | Pass   |
| UT03    | Property Module     | Add property          | Add new property listing      | Title, Price, Location         | Property saved                | Saved successfully   | Pass   |
| UT04    | Search Module       | Search property       | Search by location            | City: Lahore                   | Relevant properties displayed | Results shown        | Pass   |
| UT05    | Notification Module | Send email            | Notify user                   | Email + Message                | Email sent                    | Sent successfully    | Pass   |

### 6.1.3 Functional Testing

Functional testing focuses on verifying that all features of the system operate according to the specified requirements. It ensures that the system behaves correctly from the user's perspective and that each function produces the expected output for given inputs.

In the Real Estate Management System, functional testing was performed on major features such as user login, property browsing, property searching, and booking functionality. Different user roles, including buyers and sellers, were tested to ensure that each role has access to the correct features and functionalities.

This type of testing is essential to confirm that the system meets business requirements and delivers the expected services to users. It also helps identify issues related to incorrect logic, missing features, or improper handling of user inputs. The following table illustrates the functional testing cases executed during this phase.

**Table 6.2: Functional Testing**

| <b>Test ID</b> | <b>Module Name</b> | <b>Test Case/Test Script</b> | <b>Attribute &amp; Value</b> | <b>Expected Result</b>     | <b>Actual Result</b> | <b>Status</b> |
|----------------|--------------------|------------------------------|------------------------------|----------------------------|----------------------|---------------|
| FT01           | Login Module       | Login as Buyer               | Username, Password           | Buyer dashboard displayed  | Loaded successfully  | Pass          |
| FT02           | Login Module       | Login as Seller              | Credentials                  | Seller dashboard displayed | Loaded successfully  | Pass          |
| FT03           | Property Module    | View properties              | Select category              | Property list displayed    | Displayed correctly  | Pass          |
| FT04           | Search Module      | Search property              | Location filter              | Filtered results shown     | Results correct      | Pass          |
| FT05           | Booking Module     | Book property                | Property + User              | Booking confirmed          | Confirmed            | Pass          |

#### **6.1.4 Integration Testing**

Integration testing is performed to ensure that different modules of the system work together seamlessly. After individual modules have been tested, they are combined and tested as a group to verify proper interaction and data flow between them.

In this system, integration testing was conducted on combinations of modules such as login with profile management, property handling with database storage, search functionality with map integration, and booking with payment processing. This testing ensured that data is correctly transferred between modules and that there are no communication errors.

Integration testing is crucial for identifying issues that may arise due to module dependencies, such as incorrect data exchange or system crashes. It ensures that the system functions smoothly as a complete application. The following table presents the integration testing results.

**Table 6.3: Integration Testing**

| <b>Test ID</b> | <b>Modules Involved</b> | <b>Test Case/Test Script</b> | <b>Attribute &amp; Value</b> | <b>Expected Result</b>                       | <b>Actual Result</b>   | <b>Status</b> |
|----------------|-------------------------|------------------------------|------------------------------|----------------------------------------------|------------------------|---------------|
| IT01           | Login & Profile         | Login and view profile       | Credentials                  | Profile displayed after login                | Displayed successfully | Pass          |
| IT02           | Property & Database     | Add property                 | Property data                | Property stored in database                  | Stored successfully    | Pass          |
| IT03           | Search & Map            | Search property              | Location input               | Map displays relevant properties             | Correct results shown  | Pass          |
| IT04           | Booking & Notification  | Book property                | Property + User              | Booking confirmed and notification sent      | Successfully completed | Pass          |
| IT05           | Admin & Property Module | Approve property             | Property ID                  | Property status updated and visible to users | Approved successfully  | Pass          |

## 6.2 Automated Testing

Automated testing involves the use of specialized tools and scripts to execute test cases automatically. It is particularly useful for repetitive tasks, regression testing, and large-scale systems where manual testing would be time-consuming and error-prone. Automated testing ensures consistency, reduces human effort, and increases testing efficiency.

In this project, several automated testing tools were used to validate different components of the system. Jest and Mocha were used for backend and API testing to ensure that business logic and data operations function correctly. Postman was used to test API endpoints and validate responses, ensuring proper communication between frontend and backend. Selenium was used for frontend testing to simulate user interactions such as navigation, form submission, and button clicks.

These tools helped ensure that the system performs accurately under various conditions and that all components function correctly together. Automated testing also supports continuous testing and future system scalability. The following table summarizes the automated testing tools and their results

**Table 6.4: Automated Testing**

| <b>Tool</b>            | <b>Applied On</b> | <b>Test Cases/Requirements Tested</b> | <b>Results</b> |
|------------------------|-------------------|---------------------------------------|----------------|
| Jest                   | Backend Modules   | Login, Profile, Property              | Pass           |
| Postman                | API Endpoints     | CRUD operations, response validation  | Pass           |
| Selenium               | Frontend          | UI navigation, forms                  | Pass           |
| Google Chrome DevTools | Frontend          | Debugging, UI issues, performance     | Pass           |



# **Chapter 7**

## **CONCLUSION**

## 7. Conclusion

The Real Estate Management System (Apni Jgah) has been successfully designed, developed, and implemented as a modern, intelligent, and user-centered web-based platform that aims to simplify and digitalize property-related activities. The system effectively addresses the limitations of traditional real estate practices, such as lack of transparency, scattered property information, dependency on agents, and inefficient communication between buyers and sellers.

The primary objective of this project was to provide a centralized platform where users can easily search, analyze, and manage property listings in a structured and reliable manner. This objective has been successfully achieved through the integration of essential modules including user registration and authentication, property listing management, advanced search and filtering, favorites management, chatbot support, and an administrative control panel. Each module has been carefully designed to ensure smooth interaction between users and the system while maintaining data accuracy and consistency.

One of the key strengths of this system is the integration of intelligent features, particularly the AI-based price prediction functionality. This feature enhances the overall value of the platform by providing users with an estimated understanding of property prices based on available data. It assists users in making informed decisions and reduces the risk of overpricing or underpricing in the real estate market. In addition, the chatbot feature improves user interaction by providing instant responses to common queries, making the system more interactive and user-friendly.

From a technical perspective, the system has been developed using modern web technologies, ensuring scalability, reliability, and performance. The backend is implemented using Django, which efficiently handles server-side logic, authentication, and database operations, while the frontend is designed using HTML, CSS, and JavaScript to provide a responsive and intuitive user interface. The database system ensures secure storage and quick retrieval of data, supporting smooth system operations even with increasing data volume.

The system has undergone comprehensive testing, including unit testing, functional testing, integration testing, and system testing, to ensure that all components function correctly and meet the specified requirements. Testing results indicate that the system is stable, efficient, and capable of handling real-world scenarios. Errors identified during testing were resolved, leading to improved system reliability and performance.

Another important achievement of this project is the improvement in transparency and trust within the real estate process. By providing verified property listings, structured data management, and administrative monitoring, the system reduces the chances of fraudulent activities and misleading

information. Users can rely on accurate and updated data, which significantly enhances their confidence in the platform.

Furthermore, the system improves overall efficiency by reducing manual effort and eliminating the need for multiple platforms. Buyers can easily explore properties, compare options, and make decisions, while sellers can manage their listings effectively. Administrators have complete control over system operations, ensuring proper monitoring and maintenance.

In conclusion, the Apni Jgah Real Estate Management System successfully demonstrates how modern technologies and intelligent features can be utilized to transform traditional real estate processes into a digital, efficient, and user-friendly system. The project not only fulfills academic requirements but also has strong potential for real-world implementation. With further enhancements and scalability, it can evolve into a full-scale commercial platform that significantly impacts the real estate industry.

## **7.1 Future Work**

While the current version of the Real Estate Management System successfully covers essential functionalities, there are several opportunities for future improvements:

### **7.1.1 Advanced Property Recommendations**

In future versions, machine learning models can be integrated to recommend properties based on user preferences, search history, and behavior. This will improve user experience and decision-making.

### **7.1.2 Mobile Application Development**

A dedicated mobile application (Android/iOS) can be developed to provide better accessibility and allow users to browse and manage properties on the go.

### **7.1.3 Virtual Property Tours**

The system can include 3D virtual tours or video walkthroughs of properties, enabling users to explore properties remotely without visiting physically.

### **7.1.4 Multi-City and International Expansion**

The platform can be extended to support multiple cities and countries, allowing users to search and list properties globally with localized features.

### **7.1.5 Subscription Plans**

Premium subscription models can be introduced for agents and sellers, offering benefits such as featured listings, higher visibility, and advanced analytics.

#### **7.1.6 Advanced Analytics and Dashboard**

Future updates can include dashboards that provide insights into property trends, user behavior, and sales performance, helping administrators and agents make better decisions.

#### **7.1.7 Enhanced Security Features**

Security can be improved by implementing multi-factor authentication, data encryption, and secure payment processing to protect user information and transactions.

By implementing these enhancements, the Real Estate Management System can evolve into a fully advanced digital platform, providing smarter, faster, and more secure real estate services for users worldwide.

## 8. References

- [1] N. Kok and M. Monkkonen, "Real estate market analysis and property valuation using digital platforms," *Journal of Real Estate Research*, vol. 43, no. 2, pp. 123–140, 2021.
- [2] A. Gupta and R. Sharma, "Design and development of a web-based real estate management system," *International Journal of Computer Science and Information Technologies*, vol. 12, no. 4, pp. 210–218, 2022.
- [3] S. Patel, "Online property search systems and user experience optimization," *International Journal of Web Engineering*, vol. 10, no. 3, pp. 95–105, 2021.
- [4] Python Software Foundation, "Python Documentation," [Online]. Available: <https://docs.python.org/3/> [Accessed: Apr. 2026].
- [5] Django, "Django Documentation," [Online]. Available: <https://docs.djangoproject.com/> [Accessed: Apr. 2026].
- [6] SQLite, "SQLite Documentation," [Online]. Available: <https://www.sqlite.org/docs.html> [Accessed: Apr. 2026].
- [7] Google, "Google Maps Platform Documentation," [Online]. Available: <https://developers.google.com/maps/documentation> [Accessed: Apr. 2026].
- [8] OpenStreetMap Foundation, "OpenStreetMap API Documentation," [Online]. Available: <https://wiki.openstreetmap.org/wiki/API> [Accessed: Apr. 2026].
- [9] Stripe Inc., "Stripe Payment API Documentation," [Online]. Available: <https://stripe.com/docs> [Accessed: Apr. 2026].
- [10] Zameen Media Pvt. Ltd., "Zameen.com – Pakistan Real Estate Platform," [Online]. Available: <https://www.zameen.com/> [Accessed: Apr. 2026].
- [11] Graana Pvt. Ltd., "Graana.com – Smart Real Estate Platform," [Online]. Available: <https://www.graana.com/> [Accessed: Apr. 2026].
- [12] J. Brown and K. Wilson, "Machine learning approaches for property price prediction," *Journal of Data Analytics*, vol. 8, no. 1, pp. 45–60, 2022.